🏗️ TallerHub — Documentación de Implementación y Estructura de Código

Versión: 1.0.0
Fecha: 13 de marzo de 2026
Clasificación: Documento de Implementación — Fase 2
Prerrequisito: 01-arquitectura.md (./01-arquitectura.md) (Fase 1)
Estado: Aprobado para desarrollo

Tabla de Contenidos

1. Estructura Completa de Monorepo
2. Arquitectura Backend NestJS
3. Arquitectura Frontend Next.js
4. Arquitectura App Móvil React Native/Expo
5. Convenciones de Código
6. Patrón de Carpetas Detallado
7. Estrategia Multi-Tenant en Código
8. Estrategia RBAC en Código
9. Guías de Desarrollo

---

1. Estructura Completa de Monorepo

1.1 Decisión: Monorepo con Turborepo

Se utiliza un **monorepo gestionado con Turborepo** para compartir tipos, configuraciones y pipelines de build entre backend, frontend y mobile.

Justificación:

Criterio	Monorepo	Multi-repo
Tipos compartidos	✅ Un solo `@tallerhub/shared`	❌ Publicar paquete por cada cambio
Refactoring cross-stack	✅ Atómico	❌ PRs coordinados
CI/CD	✅ Un pipeline con cache	⚠️ Múltiples pipelines
Onboarding desarrollador	✅ Un solo `git clone`	❌ Múltiples repos
Complejidad	⚠️ Requiere tooling	✅ Simple por repo

1.2 Árbol de Carpetas Raíz

```
tallerhub/
├── apps/
│   ├── backend/                  # API NestJS
│   ├── frontend/                 # Panel admin Next.js 14
│   ├── mobile/                   # App técnicos React Native/Expo
│   └── portal/                   # Portal cliente Next.js 14 (futuro V2)
│
├── packages/
│   ├── shared/                   # Tipos, enums, constantes compartidas
│   │   ├── src/
│   │   │   ├── types/            # Interfaces TypeScript compartidas
│   │   │   ├── enums/            # Enums del dominio
│   │   │   ├── constants/        # Constantes globales
│   │   │   ├── validators/       # Schemas Zod compartidos
│   │   │   └── utils/            # Utilidades puras (formateo, cálculos)
│   │   ├── package.json
│   │   └── tsconfig.json
│   │
│   ├── ui/                       # Componentes UI compartidos (futuro)
│   │   ├── src/
│   │   └── package.json
│   │
│   ├── eslint-config/            # Configuración ESLint compartida
│   │   ├── base.js
│   │   ├── nestjs.js
│   │   ├── nextjs.js
│   │   ├── react-native.js
│   │   └── package.json
│   │
│   └── tsconfig/                 # Configuraciones TypeScript base
│       ├── base.json
│       ├── nestjs.json
│       ├── nextjs.json
│       ├── react-native.json
│       └── package.json
│
├── docker/
│   ├── backend/
│   │   ├── Dockerfile
│   │   └── Dockerfile.dev
│   ├── frontend/
│   │   ├── Dockerfile
│   │   └── Dockerfile.dev
│   ├── nginx/
│   │   ├── nginx.conf
│   │   └── Dockerfile
│   └── postgres/
│       └── init.sql
│
├── scripts/
│   ├── setup.sh                  # Setup inicial del proyecto
│   ├── seed.ts                   # Seed de datos base
│   ├── generate-module.sh        # Generador de módulos NestJS
│   ├── generate-migration.sh     # Wrapper de migraciones Prisma
│   ├── db-reset.sh               # Reset de BD para desarrollo
│   └── health-check.sh           # Verificación de servicios
│
├── .github/
│   ├── workflows/
│   │   ├── ci.yml                # Pipeline CI
│   │   ├── deploy-staging.yml    # Deploy a staging
│   │   └── deploy-production.yml # Deploy a producción
```

```
│   ├── PULL_REQUEST_TEMPLATE.md
│   └── ISSUE_TEMPLATE/
│       ├── bug_report.md
│       └── feature_request.md
│
├── docs/
│   ├── 01-arquitectura.md        # Fase 1
│   ├── 02-implementacion.md      # Este documento (Fase 2)
│   ├── 03-prisma-schema.md       # Fase 3
│   ├── API.md                    # Documentación API (generada)
│   └── CHANGELOG.md
│
├── .env.example                  # Variables de entorno de ejemplo
├── .env.development              # Variables de desarrollo (gitignored)
├── .env.test                     # Variables para testing
├── .gitignore
├── .prettierrc
├── .eslintrc.js
├── docker-compose.yml            # Stack completo de desarrollo
├── docker-compose.prod.yml       # Stack de producción
├── turbo.json                    # Configuración Turborepo
├── package.json                  # Root package.json (workspaces)
├── pnpm-workspace.yaml           # Definición de workspaces pnpm
├── tsconfig.json                 # TSConfig raíz
└── README.md
```

1.3 Configuración del Workspace (pnpm)

`pnpm-workspace.yaml`:

```yaml
packages:
  - 'apps/*'
  - 'packages/*'
```

`package.json` **(raíz):**

```json
{
  "name": "tallerhub",
  "private": true,
  "scripts": {
    "dev": "turbo run dev",
    "dev:backend": "turbo run dev --filter=@tallerhub/backend",
    "dev:frontend": "turbo run dev --filter=@tallerhub/frontend",
    "dev:mobile": "turbo run dev --filter=@tallerhub/mobile",
    "build": "turbo run build",
    "build:backend": "turbo run build --filter=@tallerhub/backend",
    "build:frontend": "turbo run build --filter=@tallerhub/frontend",
    "lint": "turbo run lint",
    "test": "turbo run test",
    "test:e2e": "turbo run test:e2e",
    "format": "prettier --write \"**/*.{ts,tsx,md,json}\"",
    "db:generate": "pnpm --filter @tallerhub/backend prisma generate",
    "db:migrate": "pnpm --filter @tallerhub/backend prisma migrate dev",
    "db:migrate:prod": "pnpm --filter @tallerhub/backend prisma migrate deploy",
    "db:seed": "pnpm --filter @tallerhub/backend prisma db seed",
    "db:studio": "pnpm --filter @tallerhub/backend prisma studio",
    "db:reset": "pnpm --filter @tallerhub/backend prisma migrate reset",
    "docker:up": "docker compose up -d",
    "docker:down": "docker compose down",
    "docker:logs": "docker compose logs -f",
    "clean": "turbo run clean && rm -rf node_modules",
    "prepare": "husky install"
  },
  "devDependencies": {
    "turbo": "^2.3.0",
    "prettier": "^3.4.0",
    "husky": "^9.1.0",
    "lint-staged": "^15.3.0",
    "@commitlint/cli": "^19.6.0",
    "@commitlint/config-conventional": "^19.6.0"
  },
  "lint-staged": {
    "*.{ts,tsx}": ["eslint --fix", "prettier --write"],
    "*.{json,md}": ["prettier --write"]
  },
  "engines": {
    "node": ">=20.0.0",
    "pnpm": ">=9.0.0"
  }
}
```

1.4 Configuración Turborepo

`turbo.json`:

```json
{
  "$schema": "https://turbo.build/schema.json",
  "globalDependencies": [".env.development"],
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**", ".next/**", "!.next/cache/**"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    },
    "lint": {
      "dependsOn": ["^build"]
    },
    "test": {
      "dependsOn": ["^build"],
      "outputs": ["coverage/**"]
    },
    "test:e2e": {
      "dependsOn": ["^build"],
      "outputs": []
    },
    "clean": {
      "cache": false
    }
  }
}
```

1.5 Docker Compose para Desarrollo

`docker-compose.yml`:

```yaml
version: '3.9'

services:
  postgres:
    image: postgres:16-alpine
    container_name: tallerhub-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: tallerhub_dev
      POSTGRES_USER: tallerhub
      POSTGRES_PASSWORD: tallerhub_dev_2026
    ports:
      - '5432:5432'
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./docker/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U tallerhub -d tallerhub_dev']
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: tallerhub-redis
    restart: unless-stopped
    ports:
      - '6379:6379'
    volumes:
      - redis_data:/data
    command: redis-server --appendonly yes --maxmemory 256mb --maxmemory-policy
allkeys-lru
    healthcheck:
      test: ['CMD', 'redis-cli', 'ping']
      interval: 10s
      timeout: 5s
      retries: 5

  minio:
    image: minio/minio:latest
    container_name: tallerhub-minio
    restart: unless-stopped
    environment:
      MINIO_ROOT_USER: tallerhub_minio
      MINIO_ROOT_PASSWORD: tallerhub_minio_2026
    ports:
      - '9000:9000'
      - '9001:9001'
    volumes:
      - minio_data:/data
    command: server /data --console-address ":9001"
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:9000/minio/health/live']
      interval: 30s
      timeout: 10s
      retries: 3

  mailhog:
    image: mailhog/mailhog:latest
    container_name: tallerhub-mailhog
    restart: unless-stopped
    ports:
```

```yaml
      - '1025:1025'   # SMTP
      - '8025:8025'   # Web UI
    logging:
      driver: none

volumes:
  postgres_data:
  redis_data:
  minio_data:
```

1.6 Variables de Entorno

`.env.example`:

```
# ─── App ──────────────────────────────
NODE_ENV=development
APP_NAME=TallerHub
APP_URL=http://localhost:3000
API_URL=http://localhost:4000
API_PORT=4000
API_PREFIX=api/v1

# ─── Base de Datos ────────────────────
DATABASE_URL=postgresql://tallerhub:tallerhub_dev_2026@localhost:5432/tallerhub_dev?
schema=public
DATABASE_POOL_SIZE=10

# ─── Redis ────────────────────────────
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=
REDIS_DB=0

# ─── JWT ──────────────────────────────
JWT_SECRET=tu-secreto-super-seguro-cambiar-en-produccion
JWT_ACCESS_EXPIRATION=15m
JWT_REFRESH_EXPIRATION=7d
JWT_REFRESH_SECRET=otro-secreto-para-refresh-tokens

# ─── S3 / MinIO ───────────────────────
S3_ENDPOINT=http://localhost:9000
S3_ACCESS_KEY=tallerhub_minio
S3_SECRET_KEY=tallerhub_minio_2026
S3_BUCKET=tallerhub-dev
S3_REGION=us-east-1
S3_USE_SSL=false

# ─── Email ────────────────────────────
SMTP_HOST=localhost
SMTP_PORT=1025
SMTP_SECURE=false
SMTP_USER=
SMTP_PASS=
SMTP_FROM=noreply@tallerhub.com

# ─── Colas ────────────────────────────
BULL_REDIS_HOST=localhost
BULL_REDIS_PORT=6379

# ─── Rate Limiting ────────────────────
THROTTLE_TTL=60
THROTTLE_LIMIT=100

# ─── Logging ──────────────────────────
LOG_LEVEL=debug
LOG_FORMAT=pretty

# ─── Tenant ───────────────────────────
DEFAULT_TENANT_SLUG=demo
TENANT_HEADER=x-tenant-id
```

2. Arquitectura Backend NestJS

2.1 Estructura del Backend

```
apps/backend/
├── src/
│   ├── main.ts                          # Bootstrap de la aplicación
│   ├── app.module.ts                    # Módulo raíz
│   │
│   ├── config/                          # Configuración centralizada
│   │   ├── app.config.ts               # ConfigType con validación
│   │   ├── database.config.ts
│   │   ├── redis.config.ts
│   │   ├── jwt.config.ts
│   │   ├── s3.config.ts
│   │   ├── mail.config.ts
│   │   ├── throttle.config.ts
│   │   └── index.ts                    # Re-exports
│   │
│   ├── core/                            # Módulo core (siempre cargado)
│   │   ├── core.module.ts
│   │   │
│   │   ├── database/                   # Capa de base de datos
│   │   │   ├── prisma.module.ts
│   │   │   ├── prisma.service.ts       # PrismaClient extendido
│   │   │   └── prisma.middleware.ts    # Middleware multi-tenant
│   │   │
│   │   ├── auth/                       # Autenticación
│   │   │   ├── auth.module.ts
│   │   │   ├── auth.controller.ts
│   │   │   ├── auth.service.ts
│   │   │   ├── strategies/
│   │   │   │   ├── jwt.strategy.ts
│   │   │   │   └── jwt-refresh.strategy.ts
│   │   │   ├── guards/
│   │   │   │   ├── jwt-auth.guard.ts
│   │   │   │   ├── jwt-refresh.guard.ts
│   │   │   │   ├── tenant.guard.ts
│   │   │   │   └── permissions.guard.ts
│   │   │   ├── decorators/
│   │   │   │   ├── current-user.decorator.ts
│   │   │   │   ├── current-tenant.decorator.ts
│   │   │   │   ├── permissions.decorator.ts
│   │   │   │   ├── public.decorator.ts
│   │   │   │   └── api-auth.decorator.ts
│   │   │   ├── dto/
│   │   │   │   ├── login.dto.ts
│   │   │   │   ├── register.dto.ts
│   │   │   │   ├── refresh-token.dto.ts
│   │   │   │   ├── change-password.dto.ts
│   │   │   │   └── forgot-password.dto.ts
│   │   │   └── interfaces/
│   │   │       ├── jwt-payload.interface.ts
│   │   │       └── authenticated-user.interface.ts
│   │   │
│   │   ├── tenant/                     # Resolución y validación de tenant
│   │   │   ├── tenant.module.ts
│   │   │   ├── tenant.middleware.ts
│   │   │   ├── tenant.service.ts
│   │   │   └── tenant.context.ts       # AsyncLocalStorage
│   │   │
│   │   ├── cache/                      # Servicio de caché Redis
│   │   │   ├── cache.module.ts
│   │   │   └── cache.service.ts
│   │   │
│   │   ├── queue/                      # Configuración BullMQ
```

```
│   │   │   ├── queue.module.ts
│   │   │   └── queue.config.ts
│   │   │
│   │   ├── storage/                    # Servicio S3
│   │   │   ├── storage.module.ts
│   │   │   ├── storage.service.ts
│   │   │   └── storage.controller.ts   # Presigned URLs
│   │   │
│   │   ├── mail/                       # Servicio de email
│   │   │   ├── mail.module.ts
│   │   │   ├── mail.service.ts
│   │   │   └── templates/              # Plantillas de email
│   │   │       ├── base.hbs
│   │   │       ├── welcome.hbs
│   │   │       ├── quote-sent.hbs
│   │   │       ├── order-status.hbs
│   │   │       └── reset-password.hbs
│   │   │
│   │   ├── pdf/                        # Generación de PDFs
│   │   │   ├── pdf.module.ts
│   │   │   ├── pdf.service.ts
│   │   │   └── templates/
│   │   │       ├── work-order.hbs
│   │   │       ├── quote.hbs
│   │   │       └── report.hbs
│   │   │
│   │   ├── audit/                      # Auditoría transversal
│   │   │   ├── audit.module.ts
│   │   │   ├── audit.service.ts
│   │   │   ├── audit.interceptor.ts
│   │   │   └── audit.decorator.ts
│   │   │
│   │   ├── events/                     # Sistema de eventos internos
│   │   │   ├── events.module.ts
│   │   │   └── events.service.ts
│   │   │
│   │   ├── interceptors/
│   │   │   ├── response-transform.interceptor.ts
│   │   │   ├── logging.interceptor.ts
│   │   │   └── timeout.interceptor.ts
│   │   │
│   │   ├── filters/
│   │   │   ├── all-exceptions.filter.ts
│   │   │   ├── prisma-exception.filter.ts
│   │   │   └── validation-exception.filter.ts
│   │   │
│   │   ├── pipes/
│   │   │   ├── parse-uuid.pipe.ts
│   │   │   └── trim-strings.pipe.ts
│   │   │
│   │   └── utils/
│   │       ├── pagination.ts           # Helpers de paginación cursor-based
│   │       ├── slug.ts                 # Generación de slugs
│   │       ├── correlative.ts          # Generación de números correlativos
│   │       └── crypto.ts              # Hashing, tokens
│   │
│   ├── modules/                        # Módulos funcionales de negocio
│   │   ├── tenants/                    # Gestión de tenants (plataforma)
│   │   │   ├── tenants.module.ts
│   │   │   ├── tenants.controller.ts
│   │   │   ├── tenants.service.ts
│   │   │   ├── dto/
│   │   │   │   ├── create-tenant.dto.ts
```

```
│   │   │   │   └── update-tenant.dto.ts
│   │   │   └── tests/
│   │   │       ├── tenants.controller.spec.ts
│   │   │       └── tenants.service.spec.ts
│   │   │
│   │   ├── branches/                    # Sucursales
│   │   │   ├── branches.module.ts
│   │   │   ├── branches.controller.ts
│   │   │   ├── branches.service.ts
│   │   │   ├── dto/
│   │   │   │   ├── create-branch.dto.ts
│   │   │   │   └── update-branch.dto.ts
│   │   │   └── tests/
│   │   │       └── branches.service.spec.ts
│   │   │
│   │   ├── users/                       # Usuarios
│   │   │   ├── users.module.ts
│   │   │   ├── users.controller.ts
│   │   │   ├── users.service.ts
│   │   │   ├── dto/
│   │   │   │   ├── create-user.dto.ts
│   │   │   │   ├── update-user.dto.ts
│   │   │   │   └── invite-user.dto.ts
│   │   │   └── tests/
│   │   │       └── users.service.spec.ts
│   │   │
│   │   ├── roles/                       # Roles y permisos
│   │   │   ├── roles.module.ts
│   │   │   ├── roles.controller.ts
│   │   │   ├── roles.service.ts
│   │   │   ├── permissions.service.ts
│   │   │   ├── dto/
│   │   │   │   ├── create-role.dto.ts
│   │   │   │   └── assign-permissions.dto.ts
│   │   │   └── tests/
│   │   │       └── permissions.service.spec.ts
│   │   │
│   │   ├── customers/                   # Clientes (MVP)
│   │   │   ├── customers.module.ts
│   │   │   ├── customers.controller.ts
│   │   │   ├── customers.service.ts
│   │   │   ├── dto/
│   │   │   │   ├── create-customer.dto.ts
│   │   │   │   ├── update-customer.dto.ts
│   │   │   │   ├── query-customer.dto.ts
│   │   │   │   └── import-customers.dto.ts
│   │   │   ├── interfaces/
│   │   │   │   └── customer-filters.interface.ts
│   │   │   └── tests/
│   │   │       ├── customers.controller.spec.ts
│   │   │       └── customers.service.spec.ts
│   │   │
│   │   ├── assets/                      # Activos / Equipos (MVP)
│   │   │   ├── assets.module.ts
│   │   │   ├── assets.controller.ts
│   │   │   ├── assets.service.ts
│   │   │   ├── asset-types.controller.ts
│   │   │   ├── asset-types.service.ts
│   │   │   ├── dto/
│   │   │   │   ├── create-asset.dto.ts
│   │   │   │   ├── update-asset.dto.ts
│   │   │   │   └── create-asset-type.dto.ts
│   │   │   └── tests/
```

```
│   │   │       └── assets.service.spec.ts
│   │   │
│   │   ├── work-orders/                # Órdenes de Trabajo (MVP)
│   │   │   ├── work-orders.module.ts
│   │   │   ├── work-orders.controller.ts
│   │   │   ├── work-orders.service.ts
│   │   │   ├── work-order-tasks.controller.ts
│   │   │   ├── work-order-tasks.service.ts
│   │   │   ├── work-order-photos.controller.ts
│   │   │   ├── work-order-photos.service.ts
│   │   │   ├── work-order-products.controller.ts
│   │   │   ├── work-order-products.service.ts
│   │   │   ├── work-order-services.controller.ts
│   │   │   ├── work-order-services.service.ts
│   │   │   ├── work-order-comments.service.ts
│   │   │   ├── work-order-signatures.service.ts
│   │   │   ├── work-order-statuses.controller.ts
│   │   │   ├── work-order-statuses.service.ts
│   │   │   ├── processors/
│   │   │   │   ├── work-order-pdf.processor.ts
│   │   │   │   └── work-order-notification.processor.ts
│   │   │   ├── listeners/
│   │   │   │   └── work-order-events.listener.ts
│   │   │   ├── dto/
│   │   │   │   ├── create-work-order.dto.ts
│   │   │   │   ├── update-work-order.dto.ts
│   │   │   │   ├── change-status.dto.ts
│   │   │   │   ├── add-product.dto.ts
│   │   │   │   ├── add-service.dto.ts
│   │   │   │   ├── add-comment.dto.ts
│   │   │   │   ├── add-signature.dto.ts
│   │   │   │   ├── add-checklist.dto.ts
│   │   │   │   └── query-work-orders.dto.ts
│   │   │   ├── interfaces/
│   │   │   │   ├── work-order-filters.interface.ts
│   │   │   │   └── work-order-totals.interface.ts
│   │   │   └── tests/
│   │   │       ├── work-orders.controller.spec.ts
│   │   │       ├── work-orders.service.spec.ts
│   │   │       └── work-order-flow.e2e-spec.ts
│   │   │
│   │   ├── quotes/                     # Cotizaciones (MVP)
│   │   │   ├── quotes.module.ts
│   │   │   ├── quotes.controller.ts
│   │   │   ├── quotes.service.ts
│   │   │   ├── quote-items.service.ts
│   │   │   ├── quote-versions.service.ts
│   │   │   ├── quote-public.controller.ts  # Endpoint público (aprobación)
│   │   │   ├── processors/
│   │   │   │   └── quote-pdf.processor.ts
│   │   │   ├── dto/
│   │   │   │   ├── create-quote.dto.ts
│   │   │   │   ├── update-quote.dto.ts
│   │   │   │   ├── add-quote-item.dto.ts
│   │   │   │   ├── send-quote.dto.ts
│   │   │   │   └── approve-quote.dto.ts
│   │   │   └── tests/
│   │   │       └── quotes.service.spec.ts
│   │   │
│   │   ├── products/                   # Productos / Repuestos (MVP)
│   │   │   ├── products.module.ts
│   │   │   ├── products.controller.ts
│   │   │   ├── products.service.ts
```

```
│   │   │   ├── product-categories.controller.ts
│   │   │   ├── product-categories.service.ts
│   │   │   ├── dto/
│   │   │   │   ├── create-product.dto.ts
│   │   │   │   ├── update-product.dto.ts
│   │   │   │   ├── query-products.dto.ts
│   │   │   │   └── create-category.dto.ts
│   │   │   └── tests/
│   │   │       └── products.service.spec.ts
│   │   │
│   │   ├── inventory/                  # Inventario (MVP)
│   │   │   ├── inventory.module.ts
│   │   │   ├── inventory.controller.ts
│   │   │   ├── inventory.service.ts
│   │   │   ├── inventory-movements.service.ts
│   │   │   ├── dto/
│   │   │   │   ├── adjust-stock.dto.ts
│   │   │   │   ├── transfer-stock.dto.ts
│   │   │   │   └── query-inventory.dto.ts
│   │   │   └── tests/
│   │   │       └── inventory.service.spec.ts
│   │   │
│   │   ├── services/                   # Servicios / Mano de obra (MVP)
│   │   │   ├── services.module.ts
│   │   │   ├── services.controller.ts
│   │   │   ├── services.service.ts
│   │   │   ├── service-categories.controller.ts
│   │   │   ├── service-categories.service.ts
│   │   │   ├── dto/
│   │   │   │   ├── create-service.dto.ts
│   │   │   │   └── update-service.dto.ts
│   │   │   └── tests/
│   │   │       └── services.service.spec.ts
│   │   │
│   │   ├── reports/                    # Dashboard y reportes (MVP básico)
│   │   │   ├── reports.module.ts
│   │   │   ├── reports.controller.ts
│   │   │   ├── reports.service.ts
│   │   │   ├── dto/
│   │   │   │   └── report-filters.dto.ts
│   │   │   └── tests/
│   │   │       └── reports.service.spec.ts
│   │   │
│   │   ├── suppliers/                  # Proveedores (V2)
│   │   ├── purchases/                  # Compras (V2)
│   │   ├── preventive/                 # Mantenimiento preventivo (V2)
│   │   ├── schedule/                   # Agenda (V2)
│   │   ├── field-service/              # Terreno (V3)
│   │   ├── portal/                     # Portal cliente (V2)
│   │   ├── notifications/              # Notificaciones (V2)
│   │   └── integrations/               # Webhooks / API (V3)
│   │
│   └── prisma/
│       ├── schema.prisma               # Schema completo
│       ├── migrations/                 # Migraciones generadas
│       ├── seed.ts                     # Seed principal
│       └── seeders/
│           ├── permissions.seeder.ts
│           ├── roles.seeder.ts
│           ├── work-order-statuses.seeder.ts
│           ├── demo-tenant.seeder.ts
│           └── index.ts
│
```

```
├── test/
│   ├── jest-e2e.json
│   ├── setup.ts                        # Setup global de tests
│   ├── helpers/
│   │   ├── test-app.factory.ts         # Crea app de test con DB
│   │   ├── auth.helper.ts             # Genera tokens de test
│   │   ├── tenant.helper.ts           # Crea tenants de test
│   │   └── fixtures/                  # Datos de prueba
│   │       ├── customers.fixture.ts
│   │       ├── work-orders.fixture.ts
│   │       └── products.fixture.ts
│   └── e2e/
│       ├── auth.e2e-spec.ts
│       ├── customers.e2e-spec.ts
│       ├── work-orders.e2e-spec.ts
│       └── multi-tenant.e2e-spec.ts
│
├── nest-cli.json
├── tsconfig.json
├── tsconfig.build.json
├── package.json
├── jest.config.ts
└── .env.test
```

2.2 Bootstrap de la Aplicación

`src/main.ts`:

```
import { NestFactory } from '@nestjs/core';
import { ValidationPipe, VersioningType } from '@nestjs/common';
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';
import { Logger } from 'nestjs-pino';
import helmet from 'helmet';
import compression from 'compression';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    bufferLogs: true,
  });

  // Logger estructurado (Pino)
  app.useLogger(app.get(Logger));

  // Seguridad
  app.use(helmet());
  app.use(compression());
  app.enableCors({
    origin: [
      process.env.APP_URL,
      /\.tallerhub\.com$/,
    ],
    credentials: true,
  });

  // Prefijo global y versionado
  app.setGlobalPrefix(process.env.API_PREFIX || 'api/v1');
  app.enableVersioning({
    type: VersioningType.URI,
    defaultVersion: '1',
  });

  // Validación global
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
      transformOptions: {
        enableImplicitConversion: true,
      },
    }),
  );

  // Swagger (solo development/staging)
  if (process.env.NODE_ENV !== 'production') {
    const config = new DocumentBuilder()
      .setTitle('TallerHub API')
      .setDescription('API para gestión de talleres y servicios técnicos')
      .setVersion('1.0')
      .addBearerAuth()
      .addApiKey({ type: 'apiKey', name: 'x-tenant-id', in: 'header' }, 'tenant')
      .build();

    const document = SwaggerModule.createDocument(app, config);
    SwaggerModule.setup('docs', app, document);
  }

  const port = process.env.API_PORT || 4000;
  await app.listen(port);
```

```
  console.log(`🚀 TallerHub API corriendo en: http://localhost:${port}`);
  console.log(`📚 Swagger docs: http://localhost:${port}/docs`);
}

bootstrap();
```

2.3 Módulo Raíz

`src/app.module.ts`:

```
import { Module, MiddlewareConsumer, NestModule } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { ThrottlerModule } from '@nestjs/throttler';
import { LoggerModule } from 'nestjs-pino';
import { EventEmitterModule } from '@nestjs/event-emitter';

// Core
import { CoreModule } from './core/core.module';
import { TenantMiddleware } from './core/tenant/tenant.middleware';

// Módulos de negocio
import { TenantsModule } from './modules/tenants/tenants.module';
import { BranchesModule } from './modules/branches/branches.module';
import { UsersModule } from './modules/users/users.module';
import { RolesModule } from './modules/roles/roles.module';
import { CustomersModule } from './modules/customers/customers.module';
import { AssetsModule } from './modules/assets/assets.module';
import { WorkOrdersModule } from './modules/work-orders/work-orders.module';
import { QuotesModule } from './modules/quotes/quotes.module';
import { ProductsModule } from './modules/products/products.module';
import { InventoryModule } from './modules/inventory/inventory.module';
import { ServicesModule } from './modules/services/services.module';
import { ReportsModule } from './modules/reports/reports.module';

// Configuración
import {
  appConfig,
  databaseConfig,
  redisConfig,
  jwtConfig,
  s3Config,
  mailConfig,
  throttleConfig,
} from './config';

@Module({
  imports: [
    // Configuración global
    ConfigModule.forRoot({
      isGlobal: true,
      load: [appConfig, databaseConfig, redisConfig, jwtConfig, s3Config, mailConfig,
throttleConfig],
      envFilePath: [`.env.${process.env.NODE_ENV}`, '.env'],
    }),

    // Logger estructurado
    LoggerModule.forRoot({
      pinoHttp: {
        level: process.env.LOG_LEVEL || 'info',
        transport:
          process.env.NODE_ENV !== 'production'
            ? { target: 'pino-pretty', options: { colorize: true } }
            : undefined,
        redact: ['req.headers.authorization'],
      },
    }),

    // Rate limiting global
    ThrottlerModule.forRoot([{
      ttl: parseInt(process.env.THROTTLE_TTL || '60') * 1000,
      limit: parseInt(process.env.THROTTLE_LIMIT || '100'),
    }]),
```

```
    // Eventos internos
    EventEmitterModule.forRoot({
      wildcard: true,
      delimiter: '.',
      maxListeners: 20,
    }),

    // Core (DB, Auth, Cache, Queue, Storage, Mail, PDF, Audit)
    CoreModule,

    // Módulos de negocio MVP
    TenantsModule,
    BranchesModule,
    UsersModule,
    RolesModule,
    CustomersModule,
    AssetsModule,
    WorkOrdersModule,
    QuotesModule,
    ProductsModule,
    InventoryModule,
    ServicesModule,
    ReportsModule,
  ],
})
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    consumer
      .apply(TenantMiddleware)
      .exclude(
        'api/v1/auth/login',
        'api/v1/auth/register',
        'api/v1/auth/forgot-password',
        'api/v1/health',
        'api/v1/quotes/public/(.*)',
        'docs(.*)',
      )
      .forRoutes('*');
  }
}
```

2.4 Servicio Prisma Extendido

`src/core/database/prisma.service.ts`:

```
import { Injectable, OnModuleInit, OnModuleDestroy, Logger } from '@nestjs/common';
import { PrismaClient, Prisma } from '@prisma/client';

@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit, OnModuleDes-
troy {
  private readonly logger = new Logger(PrismaService.name);

  constructor() {
    super({
      log: [
        { emit: 'event', level: 'query' },
        { emit: 'event', level: 'error' },
        { emit: 'event', level: 'warn' },
      ],
    });
  }

  async onModuleInit() {
    await this.$connect();
    this.logger.log('✅ Conectado a PostgreSQL');

    // Logging de queries lentas (>200ms)
    this.$on('query' as never, (event: Prisma.QueryEvent) => {
      if (event.duration > 200) {
        this.logger.warn(
          `Query lenta (${event.duration}ms): ${event.query.substring(0, 200)}`,
        );
      }
    });
  }

  async onModuleDestroy() {
    await this.$disconnect();
  }

  /**
   * Ejecuta una operación dentro de una transacción.
   * Inyecta automáticamente el Prisma transactional client.
   */
  async executeInTransaction<T>(
    fn: (tx: Prisma.TransactionClient) => Promise<T>,
    options?: { maxWait?: number; timeout?: number },
  ): Promise<T> {
    return this.$transaction(fn, {
      maxWait: options?.maxWait || 5000,
      timeout: options?.timeout || 10000,
    });
  }

  /**
   * Extensión para soft delete.
   * Filtra automáticamente registros con deleted_at != null en findMany/findFirst.
   */
  withSoftDelete() {
    return this.$extends({
      query: {
        $allModels: {
          async findMany({ model, operation, args, query }) {
            args.where = { ...args.where, deletedAt: null };
            return query(args);
          },
```

```
          async findFirst({ model, operation, args, query }) {
            args.where = { ...args.where, deletedAt: null };
            return query(args);
          },
          async findUnique({ model, operation, args, query }) {
            return query(args);
          },
        },
      },
    });
  }
}
```

2.5 Estructura de un DTO con Validación

`src/modules/work-orders/dto/create-work-order.dto.ts`:

```
import {
  IsUUID,
  IsString,
  IsOptional,
  IsEnum,
  IsBoolean,
  IsDateString,
  MaxLength,
  ValidateNested,
} from 'class-validator';
import { Type } from 'class-transformer';
import { ApiProperty, ApiPropertyOptional } from '@nestjs/swagger';
import { WorkOrderPriority } from '@tallerhub/shared';

export class CreateWorkOrderDto {
  @ApiProperty({ description: 'ID del cliente' })
  @IsUUID()
  customerId: string;

  @ApiPropertyOptional({ description: 'ID del activo asociado' })
  @IsOptional()
  @IsUUID()
  assetId?: string;

  @ApiProperty({ description: 'ID de la sucursal' })
  @IsUUID()
  branchId: string;

  @ApiPropertyOptional({ description: 'Prioridad de la orden' })
  @IsOptional()
  @IsEnum(WorkOrderPriority)
  priority?: WorkOrderPriority;

  @ApiPropertyOptional({ description: 'Tipo de orden (reparación, diagnóstico,
etc.)' })
  @IsOptional()
  @IsString()
  @MaxLength(50)
  orderType?: string;

  @ApiPropertyOptional({ description: 'Canal de ingreso' })
  @IsOptional()
  @IsString()
  @MaxLength(50)
  channel?: string;

  @ApiPropertyOptional({ description: 'Técnico asignado' })
  @IsOptional()
  @IsUUID()
  assignedTo?: string;

  @ApiPropertyOptional({ description: 'Diagnóstico inicial / motivo de ingreso' })
  @IsOptional()
  @IsString()
  @MaxLength(5000)
  initialDiagnosis?: string;

  @ApiPropertyOptional({ description: 'Notas internas' })
  @IsOptional()
  @IsString()
  @MaxLength(5000)
  internalNotes?: string;
```

```
  @ApiPropertyOptional({ description: 'Notas visibles al cliente' })
  @IsOptional()
  @IsString()
  @MaxLength(5000)
  clientNotes?: string;

  @ApiPropertyOptional({ description: 'Fecha promesa de entrega' })
  @IsOptional()
  @IsDateString()
  promisedAt?: string;

  @ApiPropertyOptional({ description: 'Es servicio a domicilio' })
  @IsOptional()
  @IsBoolean()
  isFieldService?: boolean;

  @ApiPropertyOptional({ description: 'Dirección del servicio a domicilio' })
  @IsOptional()
  @IsString()
  fieldAddress?: string;
}
```

2.6 Interceptor de Transformación de Respuesta

`src/core/interceptors/response-transform.interceptor.ts`:

```
import {
  Injectable,
  NestInterceptor,
  ExecutionContext,
  CallHandler,
} from '@nestjs/common';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';

export interface ApiResponse<T> {
  success: boolean;
  data: T;
  meta?: {
    page?: number;
    limit?: number;
    total?: number;
    cursor?: string;
  };
  timestamp: string;
}

@Injectable()
export class ResponseTransformInterceptor<T>
  implements NestInterceptor<T, ApiResponse<T>>
{
  intercept(
    context: ExecutionContext,
    next: CallHandler,
  ): Observable<ApiResponse<T>> {
    return next.handle().pipe(
      map((data) => {
        // Si ya es una respuesta formateada, no transformar
        if (data?.success !== undefined) return data;

        // Si es paginado, extraer meta
        if (data?.items && data?.meta) {
          return {
            success: true,
            data: data.items,
            meta: data.meta,
            timestamp: new Date().toISOString(),
          };
        }

        return {
          success: true,
          data,
          timestamp: new Date().toISOString(),
        };
      }),
    );
  }
}
```

2.7 Exception Filters

`src/core/filters/all-exceptions.filter.ts` :

```
import {
  ExceptionFilter,
  Catch,
  ArgumentsHost,
  HttpException,
  HttpStatus,
  Logger,
} from '@nestjs/common';
import { Request, Response } from 'express';

@Catch()
export class AllExceptionsFilter implements ExceptionFilter {
  private readonly logger = new Logger(AllExceptionsFilter.name);

  catch(exception: unknown, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();
    const request = ctx.getRequest<Request>();

    let status = HttpStatus.INTERNAL_SERVER_ERROR;
    let message = 'Error interno del servidor';
    let errors: any = undefined;

    if (exception instanceof HttpException) {
      status = exception.getStatus();
      const exResponse = exception.getResponse();

      if (typeof exResponse === 'string') {
        message = exResponse;
      } else if (typeof exResponse === 'object') {
        message = (exResponse as any).message || message;
        errors = (exResponse as any).errors;
      }
    }

    // Log de errores 5xx
    if (status >= 500) {
      this.logger.error(
        `${request.method} ${request.url} - ${status}`,
        exception instanceof Error ? exception.stack : String(exception),
      );
    }

    response.status(status).json({
      success: false,
      error: {
        statusCode: status,
        message,
        errors,
        path: request.url,
        timestamp: new Date().toISOString(),
      },
    });
  }
}
```

`src/core/filters/prisma-exception.filter.ts` :

```
import { ExceptionFilter, Catch, ArgumentsHost, HttpStatus } from '@nestjs/common';
import { Prisma } from '@prisma/client';
import { Response } from 'express';

@Catch(Prisma.PrismaClientKnownRequestError)
export class PrismaExceptionFilter implements ExceptionFilter {
  catch(exception: Prisma.PrismaClientKnownRequestError, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();

    let status = HttpStatus.INTERNAL_SERVER_ERROR;
    let message = 'Error de base de datos';

    switch (exception.code) {
      case 'P2002': // Unique constraint
        status = HttpStatus.CONFLICT;
        const fields = (exception.meta?.target as string[])?.join(', ');
        message = `Ya existe un registro con ese valor para: ${fields}`;
        break;
      case 'P2025': // Record not found
        status = HttpStatus.NOT_FOUND;
        message = 'Registro no encontrado';
        break;
      case 'P2003': // Foreign key constraint
        status = HttpStatus.BAD_REQUEST;
        message = 'Referencia a registro inexistente';
        break;
      case 'P2014': // Required relation violation
        status = HttpStatus.BAD_REQUEST;
        message = 'No se puede eliminar: tiene registros relacionados';
        break;
    }

    response.status(status).json({
      success: false,
      error: {
        statusCode: status,
        message,
        code: exception.code,
        timestamp: new Date().toISOString(),
      },
    });
  }
}
```

2.8 Estrategia de Testing Backend

Tipo	Herramienta	Alcance	Ubicación
Unit Tests	Jest	Services, Utils, Pipes	`src/modules/*/tests/*.spec.ts`
Integration Tests	Jest + Prisma	Service + DB real	`src/modules/*/tests/*.spec.ts`
E2E Tests	Jest + Supertest	API HTTP completa	`test/e2e/*.e2e-spec.ts`
Security Tests	Jest	Multi-tenant isolation	`test/e2e/multi-tenant.e2e-spec.ts`

`jest.config.ts`:

```
import type { Config } from 'jest';

const config: Config = {
  moduleFileExtensions: ['js', 'json', 'ts'],
  rootDir: '.',
  testRegex: '.*\\.spec\\.ts$',
  transform: { '^.+\\.(t|j)s$': 'ts-jest' },
  collectCoverageFrom: [
    'src/**/*.ts',
    '!src/main.ts',
    '!src/**/*.module.ts',
    '!src/**/*.dto.ts',
    '!src/**/*.interface.ts',
    '!src/prisma/**',
  ],
  coverageDirectory: './coverage',
  testEnvironment: 'node',
  moduleNameMapper: {
    '^@/(.*)$': '<rootDir>/src/$1',
    '^@tallerhub/shared$': '<rootDir>/../../packages/shared/src',
  },
};

export default config;
```

3. Arquitectura Frontend Next.js

3.1 Estructura del Frontend

```
apps/frontend/
├── src/
│   ├── app/                              # App Router (Next.js 14)
│   │   ├── layout.tsx                    # Layout raíz (providers globales)
│   │   ├── page.tsx                      # Redirect a /dashboard o /login
│   │   ├── loading.tsx                   # Loading global
│   │   ├── error.tsx                     # Error boundary global
│   │   ├── not-found.tsx                 # 404 personalizado
│   │   ├── globals.css                   # Estilos globales + Tailwind
│   │   │
│   │   ├── (auth)/                       # Grupo: páginas públicas
│   │   │   ├── layout.tsx                # Layout auth (centrado, sin sidebar)
│   │   │   ├── login/
│   │   │   │   └── page.tsx
│   │   │   ├── register/
│   │   │   │   └── page.tsx
│   │   │   ├── forgot-password/
│   │   │   │   └── page.tsx
│   │   │   └── reset-password/
│   │   │       └── page.tsx
│   │   │
│   │   ├── (dashboard)/                  # Grupo: panel principal (autenticado)
│   │   │   ├── layout.tsx                # Layout con Sidebar + Header + Breadcrumbs
│   │   │   ├── dashboard/
│   │   │   │   ├── page.tsx              # Dashboard principal
│   │   │   │   └── loading.tsx
│   │   │   │
│   │   │   ├── customers/
│   │   │   │   ├── page.tsx              # Lista de clientes
│   │   │   │   ├── new/
│   │   │   │   │   └── page.tsx          # Crear cliente
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── page.tsx          # Detalle del cliente
│   │   │   │   │   ├── edit/
│   │   │   │   │   │   └── page.tsx      # Editar cliente
│   │   │   │   │   ├── assets/
│   │   │   │   │   │   └── page.tsx      # Activos del cliente
│   │   │   │   │   └── history/
│   │   │   │   │       └── page.tsx      # Historial del cliente
│   │   │   │   └── import/
│   │   │   │       └── page.tsx          # Importación masiva
│   │   │   │
│   │   │   ├── work-orders/
│   │   │   │   ├── page.tsx              # Lista de OT (con filtros y Kanban)
│   │   │   │   ├── new/
│   │   │   │   │   └── page.tsx          # Crear OT
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── page.tsx          # Detalle de OT (vista completa)
│   │   │   │   │   ├── edit/
│   │   │   │   │   │   └── page.tsx      # Editar OT
│   │   │   │   └── statuses/
│   │   │   │       └── page.tsx          # Configurar estados
│   │   │   │
│   │   │   ├── quotes/
│   │   │   │   ├── page.tsx              # Lista de cotizaciones
│   │   │   │   ├── new/
│   │   │   │   │   └── page.tsx
│   │   │   │   └── [id]/
│   │   │   │       ├── page.tsx          # Detalle cotización
│   │   │   │       └── edit/
│   │   │   │           └── page.tsx
│   │   │   │
```

```
│   │   │   ├── products/
│   │   │   │   ├── page.tsx
│   │   │   │   ├── new/
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── [id]/
│   │   │   │   │   ├── page.tsx
│   │   │   │   │   └── edit/
│   │   │   │   │       └── page.tsx
│   │   │   │   ├── categories/
│   │   │   │   │   └── page.tsx
│   │   │   │   └── import/
│   │   │   │       └── page.tsx
│   │   │   │
│   │   │   ├── inventory/
│   │   │   │   ├── page.tsx            # Stock actual por sucursal
│   │   │   │   ├── movements/
│   │   │   │   │   └── page.tsx        # Kardex / movimientos
│   │   │   │   └── adjustments/
│   │   │   │       └── page.tsx        # Ajustes manuales
│   │   │   │
│   │   │   ├── services/
│   │   │   │   ├── page.tsx
│   │   │   │   ├── new/
│   │   │   │   │   └── page.tsx
│   │   │   │   └── [id]/
│   │   │   │       └── edit/
│   │   │   │           └── page.tsx
│   │   │   │
│   │   │   ├── reports/
│   │   │   │   ├── page.tsx            # Hub de reportes
│   │   │   │   ├── sales/
│   │   │   │   │   └── page.tsx
│   │   │   │   ├── productivity/
│   │   │   │   │   └── page.tsx
│   │   │   │   └── inventory/
│   │   │   │       └── page.tsx
│   │   │   │
│   │   │   └── settings/
│   │   │       ├── page.tsx            # Config general
│   │   │       ├── branches/
│   │   │       │   └── page.tsx
│   │   │       ├── users/
│   │   │       │   ├── page.tsx
│   │   │       │   └── [id]/
│   │   │       │       └── page.tsx
│   │   │       ├── roles/
│   │   │       │   ├── page.tsx
│   │   │       │   └── [id]/
│   │   │       │       └── page.tsx
│   │   │       └── branding/
│   │   │           └── page.tsx
│   │   │
│   │   └── api/                        # Route Handlers (BFF)
│   │       └── auth/
│   │           └── [...nextauth]/
│   │               └── route.ts
│   │
│   ├── components/                     # Componentes reutilizables
│   │   ├── ui/                         # shadcn/ui (generados)
│   │   │   ├── button.tsx
│   │   │   ├── input.tsx
│   │   │   ├── dialog.tsx
│   │   │   ├── dropdown-menu.tsx
```

```
│   │   │   ├── table.tsx
│   │   │   ├── badge.tsx
│   │   │   ├── card.tsx
│   │   │   ├── select.tsx
│   │   │   ├── tabs.tsx
│   │   │   ├── toast.tsx
│   │   │   ├── tooltip.tsx
│   │   │   ├── command.tsx              # Buscador global (⌘K)
│   │   │   ├── calendar.tsx
│   │   │   ├── popover.tsx
│   │   │   ├── sheet.tsx
│   │   │   ├── skeleton.tsx
│   │   │   └── ...
│   │   │
│   │   ├── layout/                      # Componentes de layout
│   │   │   ├── sidebar.tsx              # Sidebar principal
│   │   │   ├── header.tsx               # Header con usuario, notificaciones
│   │   │   ├── breadcrumbs.tsx
│   │   │   ├── mobile-nav.tsx
│   │   │   ├── user-menu.tsx
│   │   │   ├── notifications-popover.tsx
│   │   │   ├── branch-selector.tsx      # Selector de sucursal activa
│   │   │   └── global-search.tsx        # Buscador global (⌘K)
│   │   │
│   │   ├── shared/                      # Componentes de negocio compartidos
│   │   │   ├── data-table/              # Tabla genérica potente
│   │   │   │   ├── data-table.tsx
│   │   │   │   ├── data-table-toolbar.tsx
│   │   │   │   ├── data-table-pagination.tsx
│   │   │   │   ├── data-table-column-header.tsx
│   │   │   │   ├── data-table-faceted-filter.tsx
│   │   │   │   └── data-table-row-actions.tsx
│   │   │   ├── status-badge.tsx         # Badge de estado configurable
│   │   │   ├── priority-badge.tsx
│   │   │   ├── customer-picker.tsx      # Selector de cliente con búsqueda
│   │   │   ├── asset-picker.tsx
│   │   │   ├── product-picker.tsx
│   │   │   ├── service-picker.tsx
│   │   │   ├── user-picker.tsx
│   │   │   ├── file-uploader.tsx        # Upload con presigned URL
│   │   │   ├── image-gallery.tsx
│   │   │   ├── signature-pad.tsx        # Firma digital
│   │   │   ├── timeline.tsx             # Timeline de actividad
│   │   │   ├── confirm-dialog.tsx
│   │   │   ├── empty-state.tsx
│   │   │   ├── stat-card.tsx            # Tarjeta de KPI
│   │   │   ├── date-range-picker.tsx
│   │   │   ├── currency-input.tsx
│   │   │   └── pdf-viewer.tsx
│   │   │
│   │   └── features/                    # Componentes por feature (Client Compon-
ents)
│   │       ├── customers/
│   │       │   ├── customer-form.tsx
│   │       │   ├── customer-table.tsx
│   │       │   ├── customer-detail-tabs.tsx
│   │       │   └── customer-import-wizard.tsx
│   │       ├── work-orders/
│   │       │   ├── work-order-form.tsx
│   │       │   ├── work-order-table.tsx
│   │       │   ├── work-order-kanban.tsx
│   │       │   ├── work-order-detail.tsx
│   │       │   ├── work-order-products-tab.tsx
```

```
│   │   │   ├── work-order-services-tab.tsx
│   │   │   ├── work-order-photos-tab.tsx
│   │   │   ├── work-order-comments.tsx
│   │   │   ├── work-order-signatures.tsx
│   │   │   ├── work-order-checklist.tsx
│   │   │   ├── work-order-timeline.tsx
│   │   │   └── work-order-pdf-button.tsx
│   │   ├── quotes/
│   │   │   ├── quote-form.tsx
│   │   │   ├── quote-table.tsx
│   │   │   ├── quote-items-editor.tsx
│   │   │   ├── quote-totals.tsx
│   │   │   └── quote-send-dialog.tsx
│   │   ├── products/
│   │   │   ├── product-form.tsx
│   │   │   └── product-table.tsx
│   │   ├── inventory/
│   │   │   ├── inventory-table.tsx
│   │   │   ├── stock-adjustment-form.tsx
│   │   │   └── movement-history.tsx
│   │   ├── services/
│   │   │   ├── service-form.tsx
│   │   │   └── service-table.tsx
│   │   ├── reports/
│   │   │   ├── dashboard-stats.tsx
│   │   │   ├── orders-by-status-chart.tsx
│   │   │   ├── revenue-chart.tsx
│   │   │   └── technician-productivity.tsx
│   │   └── settings/
│   │       ├── general-settings-form.tsx
│   │       ├── branding-form.tsx
│   │       ├── user-form.tsx
│   │       └── role-permissions-matrix.tsx
│   │
│   ├── hooks/                          # Hooks personalizados
│   │   ├── use-auth.ts                  # Estado de autenticación
│   │   ├── use-tenant.ts               # Datos del tenant actual
│   │   ├── use-permissions.ts           # Verificación de permisos
│   │   ├── use-branch.ts               # Sucursal activa
│   │   ├── use-debounce.ts
│   │   ├── use-local-storage.ts
│   │   ├── use-media-query.ts
│   │   ├── use-pagination.ts
│   │   ├── use-filters.ts              # Filtros persistentes en URL
│   │   ├── use-confirmation.ts          # Dialog de confirmación
│   │   └── use-toast.ts
│   │
│   ├── contexts/                       # Contextos React
│   │   ├── auth-context.tsx             # Proveedor de autenticación
│   │   ├── tenant-context.tsx           # Proveedor de tenant
│   │   ├── branch-context.tsx           # Proveedor de sucursal activa
│   │   └── theme-context.tsx            # Tema (claro/oscuro)
│   │
│   ├── services/                       # Capa de servicios API
│   │   ├── api-client.ts               # Fetch wrapper con interceptors
│   │   ├── auth.service.ts
│   │   ├── customers.service.ts
│   │   ├── assets.service.ts
│   │   ├── work-orders.service.ts
│   │   ├── quotes.service.ts
│   │   ├── products.service.ts
│   │   ├── inventory.service.ts
│   │   ├── services.service.ts
```

```
│   │   ├── reports.service.ts
│   │   ├── users.service.ts
│   │   ├── roles.service.ts
│   │   ├── branches.service.ts
│   │   ├── storage.service.ts
│   │   └── settings.service.ts
│   │
│   ├── lib/                              # Utilidades y helpers
│   │   ├── utils.ts                      # cn() y utilidades de Tailwind
│   │   ├── format.ts                     # Formateo de moneda, fechas, etc.
│   │   ├── constants.ts                  # Constantes del frontend
│   │   ├── validations.ts               # Schemas Zod para formularios
│   │   └── navigation.ts               # Items de menú y rutas
│   │
│   ├── types/                            # Tipos del frontend
│   │   ├── api.types.ts                 # Tipos de respuesta API
│   │   ├── table.types.ts              # Tipos de DataTable
│   │   └── forms.types.ts
│   │
│   └── styles/
│       └── globals.css                   # Custom CSS + Tailwind layers
│
├── public/
│   ├── logo.svg
│   ├── logo-dark.svg
│   ├── favicon.ico
│   └── images/
│       ├── auth-bg.jpg
│       └── empty-states/
│           ├── no-customers.svg
│           ├── no-orders.svg
│           └── no-quotes.svg
│
├── next.config.js
├── tailwind.config.ts
├── postcss.config.js
├── tsconfig.json
├── package.json
├── components.json                       # Configuración shadcn/ui
└── middleware.ts                         # Middleware Next.js (auth redirect)
```

3.2 Server Components vs Client Components

Regla de decisión:

Criterio	Server Component	Client Component
Fetch de datos	✅ Siempre	❌ Nunca directamente
Layouts y páginas estáticas	✅ Por defecto	❌ No necesario
SEO / metadata	✅ generateMetadata	❌ No aplica
Interactividad (onClick, on-Change)	❌ No soportado	✅ Requerido
Estado (useState, useEffect)	❌ No soportado	✅ Requerido
Formularios	❌ Solo Server Actions	✅ React Hook Form
Tablas con sorting/filtering	❌ No soportado	✅ Requerido
Charts y gráficos	❌ No soportado	✅ Requerido

Patrón aplicado: "Server Component Wrapper + Client Component Interactivo"

```
// app/(dashboard)/customers/page.tsx — SERVER COMPONENT
import { CustomersService } from '@/services/customers.service';
import { CustomerTable } from '@/components/features/customers/customer-table';

export const metadata = { title: 'Clientes | TallerHub' };

export default async function CustomersPage({
  searchParams,
}: {
  searchParams: { page?: string; search?: string; status?: string };
}) {
  // Fetch en el servidor (rápido, sin waterfall del cliente)
  const { items, meta } = await CustomersService.getAll({
    page: Number(searchParams.page) || 1,
    search: searchParams.search,
    status: searchParams.status,
  });

  return (
    <div className="space-y-6">
      <div className="flex items-center justify-between">
        <h1 className="text-2xl font-bold">Clientes</h1>
      </div>
      {/* Client Component para interactividad */}
      <CustomerTable initialData={items} initialMeta={meta} />
    </div>
  );
}
```

3.3 Cliente API (Fetch Wrapper)

`src/services/api-client.ts`:

```
import { getSession } from '@/lib/session';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:4000/api/v1';

interface RequestOptions extends RequestInit {
  params?: Record<string, string | number | boolean | undefined>;
}

class ApiClient {
  private baseUrl: string;

  constructor(baseUrl: string) {
    this.baseUrl = baseUrl;
  }

  private async getHeaders(): Promise<HeadersInit> {
    const headers: HeadersInit = {
      'Content-Type': 'application/json',
    };

    // Server-side: obtener token de la session/cookie
    if (typeof window === 'undefined') {
      const session = await getSession();
      if (session?.accessToken) {
        headers['Authorization'] = `Bearer ${session.accessToken}`;
      }
      if (session?.tenantId) {
        headers['x-tenant-id'] = session.tenantId;
      }
    } else {
      // Client-side: obtener de localStorage
      const token = localStorage.getItem('access_token');
      const tenantId = localStorage.getItem('tenant_id');
      if (token) headers['Authorization'] = `Bearer ${token}`;
      if (tenantId) headers['x-tenant-id'] = tenantId;
    }

    return headers;
  }

  private buildUrl(path: string, params?: Record<string, any>): string {
    const url = new URL(`${this.baseUrl}${path}`);
    if (params) {
      Object.entries(params).forEach(([key, value]) => {
        if (value !== undefined && value !== null && value !== '') {
          url.searchParams.set(key, String(value));
        }
      });
    }
    return url.toString();
  }

  private async handleResponse<T>(response: Response): Promise<T> {
    if (!response.ok) {
      const error = await response.json().catch(() => ({}));

      if (response.status === 401) {
        // Intentar refresh token
        const refreshed = await this.refreshToken();
        if (!refreshed && typeof window !== 'undefined') {
          window.location.href = '/login';
        }
```

```
      }

      throw new ApiError(
        error.error?.message || 'Error en la solicitud',
        response.status,
        error.error?.errors,
      );
    }

    return response.json();
  }

  private async refreshToken(): Promise<boolean> {
    try {
      const refreshToken = localStorage.getItem('refresh_token');
      if (!refreshToken) return false;

      const res = await fetch(`${this.baseUrl}/auth/refresh`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ refreshToken }),
      });

      if (!res.ok) return false;

      const data = await res.json();
      localStorage.setItem('access_token', data.data.accessToken);
      localStorage.setItem('refresh_token', data.data.refreshToken);
      return true;
    } catch {
      return false;
    }
  }

  async get<T>(path: string, options?: RequestOptions): Promise<T> {
    const url = this.buildUrl(path, options?.params);
    const headers = await this.getHeaders();
    const response = await fetch(url, {
      ...options,
      method: 'GET',
      headers: { ...headers, ...options?.headers },
    });
    return this.handleResponse<T>(response);
  }

  async post<T>(path: string, body?: any, options?: RequestOptions): Promise<T> {
    const url = this.buildUrl(path, options?.params);
    const headers = await this.getHeaders();
    const response = await fetch(url, {
      ...options,
      method: 'POST',
      headers: { ...headers, ...options?.headers },
      body: body ? JSON.stringify(body) : undefined,
    });
    return this.handleResponse<T>(response);
  }

  async patch<T>(path: string, body?: any, options?: RequestOptions): Promise<T> {
    const url = this.buildUrl(path, options?.params);
    const headers = await this.getHeaders();
    const response = await fetch(url, {
      ...options,
      method: 'PATCH',
```

```
      headers: { ...headers, ...options?.headers },
      body: body ? JSON.stringify(body) : undefined,
    });
    return this.handleResponse<T>(response);
  }

  async delete<T>(path: string, options?: RequestOptions): Promise<T> {
    const url = this.buildUrl(path, options?.params);
    const headers = await this.getHeaders();
    const response = await fetch(url, {
      ...options,
      method: 'DELETE',
      headers: { ...headers, ...options?.headers },
    });
    return this.handleResponse<T>(response);
  }
}

export class ApiError extends Error {
  constructor(
    message: string,
    public statusCode: number,
    public errors?: any,
  ) {
    super(message);
    this.name = 'ApiError';
  }
}

export const apiClient = new ApiClient(API_URL);
```

3.4 Configuración Tailwind + shadcn/ui

`tailwind.config.ts`:

```
import type { Config } from 'tailwindcss';
import { fontFamily } from 'tailwindcss/defaultTheme';
import animate from 'tailwindcss-animate';

const config: Config = {
  darkMode: ['class'],
  content: ['./src/**/*.{ts,tsx}'],
  theme: {
    container: {
      center: true,
      padding: '2rem',
      screens: { '2xl': '1400px' },
    },
    extend: {
      colors: {
        border: 'hsl(var(--border))',
        input: 'hsl(var(--input))',
        ring: 'hsl(var(--ring))',
        background: 'hsl(var(--background))',
        foreground: 'hsl(var(--foreground))',
        primary: {
          DEFAULT: 'hsl(var(--primary))',
          foreground: 'hsl(var(--primary-foreground))',
        },
        secondary: {
          DEFAULT: 'hsl(var(--secondary))',
          foreground: 'hsl(var(--secondary-foreground))',
        },
        destructive: {
          DEFAULT: 'hsl(var(--destructive))',
          foreground: 'hsl(var(--destructive-foreground))',
        },
        muted: {
          DEFAULT: 'hsl(var(--muted))',
          foreground: 'hsl(var(--muted-foreground))',
        },
        accent: {
          DEFAULT: 'hsl(var(--accent))',
          foreground: 'hsl(var(--accent-foreground))',
        },
        card: {
          DEFAULT: 'hsl(var(--card))',
          foreground: 'hsl(var(--card-foreground))',
        },
        // Estados de OT (personalizables por tenant)
        status: {
          received: '#3B82F6',       // Azul
          reviewing: '#F59E0B',      // Amarillo
          'pending-approval': '#F97316', // Naranja
          approved: '#10B981',       // Verde
          'waiting-parts': '#8B5CF6', // Púrpura
          repairing: '#6366F1',      // Índigo
          ready: '#22C55E',          // Verde brillante
          delivered: '#6B7280',      // Gris
          cancelled: '#EF4444',      // Rojo
        },
      },
      fontFamily: {
        sans: ['Inter', ...fontFamily.sans],
      },
      keyframes: {
        'accordion-down': {
```

```
          from: { height: '0' },
          to: { height: 'var(--radix-accordion-content-height)' },
        },
        'accordion-up': {
          from: { height: 'var(--radix-accordion-content-height)' },
          to: { height: '0' },
        },
      },
      animation: {
        'accordion-down': 'accordion-down 0.2s ease-out',
        'accordion-up': 'accordion-up 0.2s ease-out',
      },
    },
  },
  plugins: [animate],
};

export default config;
```

`components.json` **(shadcn/ui):**

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "default",
  "rsc": true,
  "tsx": true,
  "tailwind": {
    "config": "tailwind.config.ts",
    "css": "src/app/globals.css",
    "baseColor": "neutral",
    "cssVariables": true
  },
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils"
  }
}
```

3.5 Middleware de Autenticación Next.js

`middleware.ts`**:**

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

const publicPaths = ['/login', '/register', '/forgot-password', '/reset-password'];
const apiPaths = ['/api/'];

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Rutas API: no interferir
  if (apiPaths.some((p) => pathname.startsWith(p))) {
    return NextResponse.next();
  }

  // Rutas públicas: permitir sin autenticación
  if (publicPaths.some((p) => pathname.startsWith(p))) {
    const token = request.cookies.get('access_token')?.value;
    // Si ya está autenticado, redirigir al dashboard
    if (token) {
      return NextResponse.redirect(new URL('/dashboard', request.url));
    }
    return NextResponse.next();
  }

  // Rutas protegidas: verificar token
  const token = request.cookies.get('access_token')?.value;
  if (!token) {
    const loginUrl = new URL('/login', request.url);
    loginUrl.searchParams.set('redirect', pathname);
    return NextResponse.redirect(loginUrl);
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/((?!_next/static|_next/image|favicon.ico|images|logo).*)'],
};
```

3.6 Estrategia de Testing Frontend

Tipo	Herramienta	Alcance
Unit Tests	Vitest + Testing Library	Hooks, utils, componentes aislados
Component Tests	Vitest + Testing Library	Componentes con mocks de API
E2E Tests	Playwright	Flujos completos en el navegador

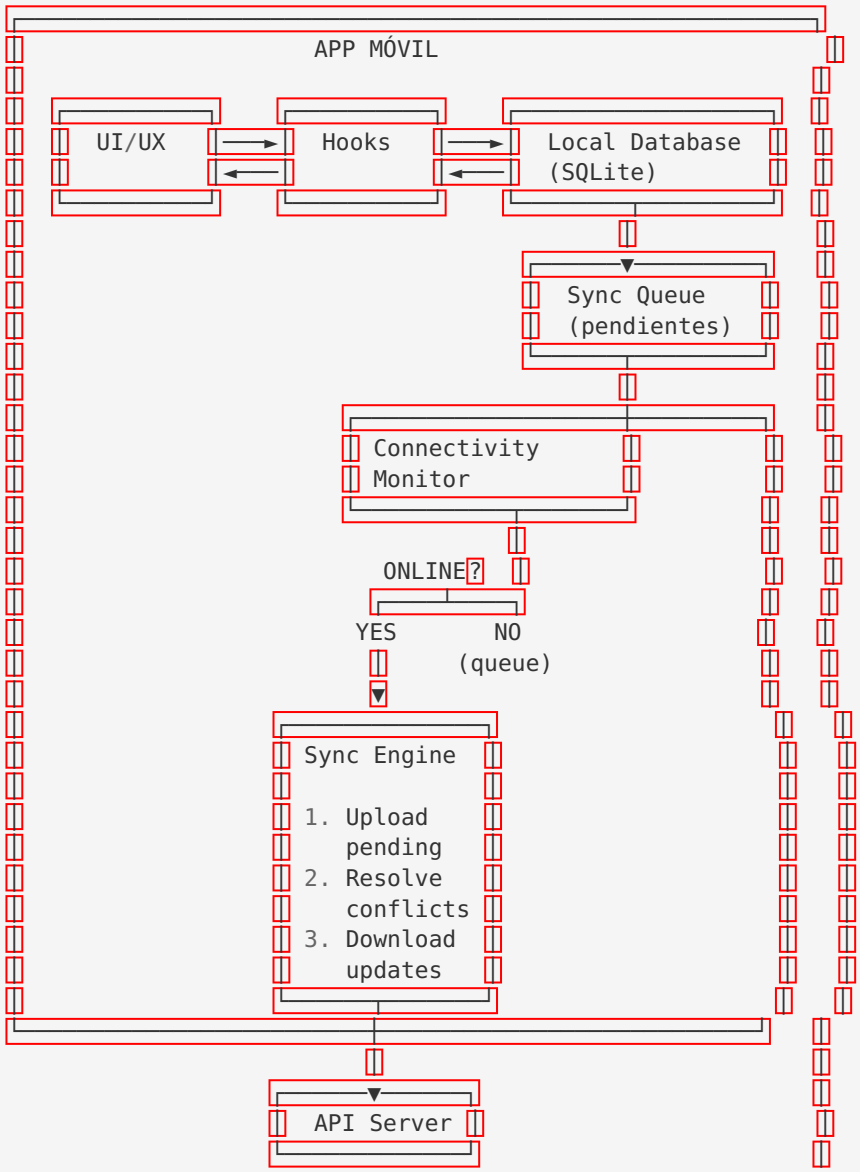
4. Arquitectura App Móvil React Native/Expo

4.1 Estructura de la App Móvil

```
apps/mobile/
├── src/
│   ├── app/                              # Expo Router (file-based routing)
│   │   ├── _layout.tsx                   # Layout raíz (providers, auth check)
│   │   ├── index.tsx                     # Splash / redirect
│   │   │
│   │   ├── (auth)/                       # Pantallas de autenticación
│   │   │   ├── _layout.tsx
│   │   │   └── login.tsx
│   │   │
│   │   └── (app)/                        # Pantallas protegidas
│   │       ├── _layout.tsx               # Tab navigator
│   │       │
│   │       ├── (tabs)/
│   │       │   ├── _layout.tsx           # Bottom tabs
│   │       │   ├── home.tsx              # Dashboard técnico
│   │       │   ├── orders/
│   │       │   │   ├── _layout.tsx       # Stack navigator
│   │       │   │   ├── index.tsx         # Lista de OT asignadas
│   │       │   │   └── [id].tsx          # Detalle de OT
│   │       │   ├── scanner.tsx           # Escáner QR
│   │       │   └── profile.tsx           # Perfil del técnico
│   │       │
│   │       ├── order-detail/
│   │       │   ├── [id]/
│   │       │   │   ├── index.tsx         # Vista completa de OT
│   │       │   │   ├── photos.tsx        # Galería y captura
│   │       │   │   ├── checklist.tsx     # Checklist interactivo
│   │       │   │   ├── products.tsx      # Repuestos usados
│   │       │   │   ├── services.tsx      # Servicios realizados
│   │       │   │   ├── comments.tsx      # Comentarios
│   │       │   │   └── signature.tsx     # Firma del cliente
│   │       │
│   │       └── field-visit/              # Servicio a domicilio (V2)
│   │           ├── [id]/
│   │           │   ├── navigate.tsx      # Navegación GPS
│   │           │   └── close.tsx         # Cierre de visita
│   │
│   ├── components/                       # Componentes reutilizables
│   │   ├── ui/                           # Componentes base
│   │   │   ├── Button.tsx
│   │   │   ├── Input.tsx
│   │   │   ├── Card.tsx
│   │   │   ├── Badge.tsx
│   │   │   ├── Loading.tsx
│   │   │   ├── EmptyState.tsx
│   │   │   ├── ErrorMessage.tsx
│   │   │   └── Avatar.tsx
│   │   │
│   │   ├── shared/                       # Componentes compartidos
│   │   │   ├── OrderCard.tsx             # Tarjeta resumen de OT
│   │   │   ├── StatusBadge.tsx
│   │   │   ├── PriorityBadge.tsx
│   │   │   ├── PhotoCapture.tsx          # Captura + galería
│   │   │   ├── SignaturePad.tsx          # Firma en pantalla
│   │   │   ├── ChecklistItem.tsx
│   │   │   ├── ProductSelector.tsx
│   │   │   ├── ServiceSelector.tsx
│   │   │   ├── QRScanner.tsx
│   │   │   └── OfflineBanner.tsx         # Indicador modo offline
│   │   │
│   │   └── layout/
```

```
│   │   ├── Header.tsx
│   │   └── SafeArea.tsx
│   │
│   ├── hooks/                          # Hooks personalizados
│   │   ├── useAuth.ts
│   │   ├── useOffline.ts               # Detección de conectividad
│   │   ├── useSync.ts                  # Sincronización
│   │   ├── useCamera.ts                # Permisos y captura
│   │   ├── useLocation.ts              # GPS
│   │   ├── useOrders.ts                # CRUD de órdenes (con cache offline)
│   │   └── useNotifications.ts         # Push notifications
│   │
│   ├── contexts/
│   │   ├── AuthContext.tsx
│   │   ├── SyncContext.tsx             # Estado de sincronización
│   │   └── ConnectivityContext.tsx
│   │
│   ├── services/                       # Servicios API
│   │   ├── api-client.ts               # Fetch wrapper mobile
│   │   ├── auth.service.ts
│   │   ├── orders.service.ts
│   │   ├── sync.service.ts             # Lógica de sincronización
│   │   └── storage.service.ts          # Upload de fotos/firma
│   │
│   ├── storage/                        # Almacenamiento local
│   │   ├── database.ts                 # SQLite / WatermelonDB
│   │   ├── schemas/                    # Schemas de tablas locales
│   │   │   ├── order.schema.ts
│   │   │   ├── customer.schema.ts
│   │   │   ├── product.schema.ts
│   │   │   └── sync-queue.schema.ts
│   │   ├── repositories/
│   │   │   ├── order.repository.ts
│   │   │   └── sync-queue.repository.ts
│   │   └── sync/
│   │       ├── sync-engine.ts          # Motor de sincronización
│   │       ├── conflict-resolver.ts    # Resolución de conflictos
│   │       └── sync-queue.ts           # Cola de operaciones pendientes
│   │
│   ├── utils/
│   │   ├── format.ts
│   │   ├── permissions.ts
│   │   └── constants.ts
│   │
│   └── types/
│       └── index.ts
│
├── assets/
│   ├── fonts/
│   ├── images/
│   └── icons/
│
├── app.json                            # Configuración Expo
├── eas.json                            # EAS Build config
├── babel.config.js
├── tsconfig.json
├── metro.config.js
└── package.json
```

4.2 Estrategia de Sincronización Offline



Estrategia de resolución de conflictos:

Escenario	Resolución
Campos simples (notas, diagnóstico)	Last-Write-Wins (timestamp del dispositivo)
Cambio de estado	Servidor tiene prioridad (puede haber sido cambiado por admin)
Fotos agregadas	Merge: se agregan todas sin conflicto
Checklist items	Merge: items marcados no se desmarcan
Repuestos / Servicios	Merge con dedup por ID
Conflicto irresolvible	Se marca para revisión manual + notificación

4.3 Configuración Expo

`app.json`:

```
{
  "expo": {
    "name": "TallerHub Técnico",
    "slug": "tallerhub-mobile",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/images/icon.png",
    "scheme": "tallerhub",
    "userInterfaceStyle": "automatic",
    "splash": {
      "image": "./assets/images/splash.png",
      "resizeMode": "contain",
      "backgroundColor": "#1E40AF"
    },
    "plugins": [
      "expo-router",
      "expo-camera",
      "expo-location",
      "expo-image-picker",
      "expo-secure-store",
      "expo-sqlite",
      [
        "expo-notifications",
        {
          "icon": "./assets/images/notification-icon.png",
          "color": "#1E40AF"
        }
      ]
    ],
    "ios": {
      "supportsTablet": false,
      "bundleIdentifier": "com.tallerhub.technician",
      "infoPlist": {
        "NSCameraUsageDescription": "TallerHub necesita acceso a la cámara para tomar
fotos de equipos y órdenes de trabajo.",
        "NSLocationWhenInUseUsageDescription": "TallerHub usa tu ubicación para regis-
trar visitas a domicilio.",
        "NSPhotoLibraryUsageDescription": "TallerHub necesita acceso a tus fotos para
adjuntar imágenes a las órdenes."
      }
    },
    "android": {
      "package": "com.tallerhub.technician",
      "adaptiveIcon": {
        "foregroundImage": "./assets/images/adaptive-icon.png",
        "backgroundColor": "#1E40AF"
      },
      "permissions": [
        "CAMERA",
        "ACCESS_FINE_LOCATION",
        "READ_EXTERNAL_STORAGE",
        "WRITE_EXTERNAL_STORAGE"
      ]
    }
  }
}
```

4.4 Manejo de Permisos del Dispositivo

```
// hooks/useCamera.ts
import { useState, useCallback } from 'react';
import * as ImagePicker from 'expo-image-picker';
import { Alert, Linking } from 'react-native';

export function useCamera() {
  const [loading, setLoading] = useState(false);

  const requestPermission = useCallback(async (): Promise<boolean> => {
    const { status } = await ImagePicker.requestCameraPermissionsAsync();
    if (status !== 'granted') {
      Alert.alert(
        'Permiso requerido',
        'TallerHub necesita acceso a la cámara para tomar fotos de los equipos.',
        [
          { text: 'Cancelar', style: 'cancel' },
          { text: 'Ir a Configuración', onPress: () => Linking.openSettings() },
        ],
      );
      return false;
    }
    return true;
  }, []);

  const takePhoto = useCallback(async () => {
    const hasPermission = await requestPermission();
    if (!hasPermission) return null;

    setLoading(true);
    try {
      const result = await ImagePicker.launchCameraAsync({
        mediaTypes: ImagePicker.MediaTypeOptions.Images,
        quality: 0.8,
        allowsEditing: false,
        exif: true, // Incluir metadata GPS
      });

      if (result.canceled) return null;
      return result.assets[0];
    } finally {
      setLoading(false);
    }
  }, [requestPermission]);

  return { takePhoto, loading };
}
```

5. Convenciones de Código

5.1 Naming Conventions

Elemento	Convención	Ejemplo
Archivos TS/TSX	`kebab-case`	`work-order-form.tsx`
Archivos de test	`*.spec.ts` / `*.e2e-spec.ts`	`customers.service.spec.ts`
Clases	`PascalCase`	`WorkOrderService`
Interfaces	`PascalCase` (sin prefijo `I`)	`AuthenticatedUser`
Types	`PascalCase`	`WorkOrderFilters`
Enums	`PascalCase` + valores `UPPER_SNAKE_CASE`	`WorkOrderPriority.HIGH`
Variables/funciones	`camelCase`	`getWorkOrderById`
Constantes	`UPPER_SNAKE_CASE`	`MAX_FILE_SIZE`
Componentes React	`PascalCase`	`CustomerTable`
Hooks	`camelCase` con prefijo `use`	`usePermissions`
Contextos	`PascalCase` con sufijo `Context`	`AuthContext`
DTOs	`PascalCase` con sufijo `Dto`	`CreateWorkOrderDto`
Guards	`PascalCase` con sufijo `Guard`	`PermissionsGuard`
Interceptors	`PascalCase` con sufijo `Interceptor`	`LoggingInterceptor`
Decorators	`PascalCase` con prefijo funcional	`@CurrentTenant()`
Tablas DB	`snake_case` (singular)	`work_order`
Columnas DB	`snake_case`	`tenant_id`
Prisma models	`PascalCase` (singular)	`WorkOrder`
API endpoints	`kebab-case` (plural)	`/work-orders`
Variables de entorno	`UPPER_SNAKE_CASE`	`DATABASE_URL`
Ramas Git	`tipo/descripcion-corta`	`feature/work-order-crud`
Commits	Conventional Commits	`feat(work-orders): add status change`

5.2 Estructura de Imports

Orden obligatorio en todo archivo TypeScript:

```
// 1. Imports de Node.js / framework
import { Injectable, Logger } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';

// 2. Imports de librerías externas
import { z } from 'zod';
import dayjs from 'dayjs';

// 3. Imports del monorepo (@tallerhub/*)
import { WorkOrderPriority } from '@tallerhub/shared';

// 4. Imports absolutos del proyecto (@/)
import { PrismaService } from '@/core/database/prisma.service';
import { CacheService } from '@/core/cache/cache.service';

// 5. Imports relativos (del mismo módulo)
import { CreateWorkOrderDto } from './dto/create-work-order.dto';
import { WorkOrderFilters } from './interfaces/work-order-filters.interface';
```

Configuración ESLint para enforce:

```
// packages/eslint-config/base.js
module.exports = {
  rules: {
    'import/order': [
      'error',
      {
        groups: [
          'builtin',
          'external',
          ['internal', 'parent', 'sibling'],
          'index',
        ],
        'newlines-between': 'always',
        alphabetize: { order: 'asc' },
      },
    ],
  },
};
```

5.3 TypeScript: Tipos, Interfaces y Enums

Regla: Interfaces para contratos de objetos, Types para uniones/intersecciones, Enums para conjuntos finitos.

```
// ─── packages/shared/src/enums/work-order.enum.ts ───
export enum WorkOrderPriority {
  LOW = 'low',
  MEDIUM = 'medium',
  HIGH = 'high',
  URGENT = 'urgent',
}

export enum WorkOrderChannel {
  IN_PERSON = 'in_person',
  PHONE = 'phone',
  WEB = 'web',
  WHATSAPP = 'whatsapp',
  EMAIL = 'email',
}

// ─── packages/shared/src/types/api.types.ts ───
export interface PaginatedResponse<T> {
  items: T[];
  meta: PaginationMeta;
}

export interface PaginationMeta {
  total: number;
  page: number;
  limit: number;
  totalPages: number;
  hasNextPage: boolean;
  hasPrevPage: boolean;
}

export interface ApiResponse<T> {
  success: boolean;
  data: T;
  meta?: PaginationMeta;
  timestamp: string;
}

export interface ApiError {
  success: false;
  error: {
    statusCode: number;
    message: string;
    errors?: Record<string, string[]>;
    path: string;
    timestamp: string;
  };
}

// ─── Union types para filtros ───
export type SortDirection = 'asc' | 'desc';
export type DateRange = { from: Date; to: Date };
```

5.4 Manejo de Errores

Backend — Excepciones de negocio personalizadas:

```
// src/core/exceptions/business.exception.ts
import { HttpException, HttpStatus } from '@nestjs/common';

export class BusinessException extends HttpException {
  constructor(message: string, statusCode: HttpStatus = HttpStatus.BAD_REQUEST) {
    super({ success: false, error: { message, statusCode } }, statusCode);
  }
}

export class TenantNotFoundException extends BusinessException {
  constructor() {
    super('Tenant no encontrado o inactivo', HttpStatus.NOT_FOUND);
  }
}

export class InsufficientStockException extends BusinessException {
  constructor(productName: string, available: number, requested: number) {
    super(
      `Stock insuficiente para "${productName}". Disponible: ${available}, solicit-
ado: ${requested}`,
      HttpStatus.CONFLICT,
    );
  }
}

export class WorkOrderCannotCloseException extends BusinessException {
  constructor(reason: string) {
    super(`No se puede cerrar la orden: ${reason}`, HttpStatus.CONFLICT);
  }
}

export class CrossTenantAccessException extends BusinessException {
  constructor() {
    super('Acceso denegado: recurso pertenece a otro tenant', HttpStatus.FORBIDDEN);
  }
}
```

5.5 Logs Estructurados

```
// Patrón de logging en servicios
@Injectable()
export class WorkOrdersService {
  private readonly logger = new Logger(WorkOrdersService.name);

  async create(tenantId: string, dto: CreateWorkOrderDto, userId: string) {
    this.logger.log({
      action: 'work_order.create',
      tenantId,
      userId,
      customerId: dto.customerId,
    });

    try {
      const order = await this.prisma.workOrder.create({ ... });

      this.logger.log({
        action: 'work_order.created',
        tenantId,
        orderId: order.id,
        orderNumber: order.orderNumber,
      });

      return order;
    } catch (error) {
      this.logger.error({
        action: 'work_order.create_failed',
        tenantId,
        userId,
        error: error.message,
      });
      throw error;
    }
  }
}
```

5.6 Commits y Versionado

Conventional Commits obligatorio:

```
<tipo>(<alcance>): <descripción>

[cuerpo opcional]

[footer opcional]
```

Tipo	Uso
`feat`	Nueva funcionalidad
`fix`	Corrección de bug
`docs`	Documentación
`style`	Formateo (sin cambios funcionales)
`refactor`	Refactoring (sin cambio funcional)
`perf`	Mejora de rendimiento
`test`	Agregar/modificar tests
`chore`	Configuración, deps, CI
`ci`	Cambios en CI/CD

Alcances válidos:

```
backend, frontend, mobile, shared, docker, ci, docs,
auth, customers, work-orders, quotes, products, inventory,
services, reports, admin, tenant, rbac
```

Ejemplos:

```
feat(work-orders): agregar cambio de estado con validación
fix(inventory): corregir cálculo de stock reservado
docs(api): documentar endpoint de cotizaciones
refactor(auth): extraer lógica de refresh token a servicio
test(customers): agregar tests e2e de CRUD
chore(docker): actualizar PostgreSQL a 16.2
```

Flujo Git (Git Flow simplificado):

```
main ──────────────────────────────────────────────►
  │                                                 │
  └──► develop ──────────────────────────────────►─┘
        │        │         │                  │
        │   feature/       │             release/
        │   work-order-   fix/            1.0.0
        │   crud         stock-calc
        │        │         │
        └────────┴─────────┘
```

---

6. Patrón de Carpetas Detallado

6.1 Backend: Ejemplo Completo de Módulo — `customers`

```
src/modules/customers/
├── customers.module.ts              # Definición del módulo NestJS
├── customers.controller.ts          # Endpoints REST
├── customers.service.ts             # Lógica de negocio
├── customer-contacts.controller.ts  # Sub-recurso: contactos
├── customer-contacts.service.ts
│
├── dto/                             # Data Transfer Objects
│   ├── create-customer.dto.ts       # Validación de creación
│   ├── update-customer.dto.ts       # PartialType del create
│   ├── query-customer.dto.ts        # Filtros de búsqueda
│   └── import-customers.dto.ts      # Para importación masiva
│
├── interfaces/                      # Contratos internos
│   └── customer-filters.interface.ts
│
└── tests/
    ├── customers.controller.spec.ts # Test unitario del controller
    ├── customers.service.spec.ts    # Test unitario del service
    └── customers.e2e-spec.ts        # Test e2e (opcional por módulo)
```

`customers.module.ts`:

```typescript
import { Module } from '@nestjs/common';
import { CustomersController } from './customers.controller';
import { CustomersService } from './customers.service';
import { CustomerContactsController } from './customer-contacts.controller';
import { CustomerContactsService } from './customer-contacts.service';

@Module({
  controllers: [CustomersController, CustomerContactsController],
  providers: [CustomersService, CustomerContactsService],
  exports: [CustomersService],
})
export class CustomersModule {}
```

`customers.controller.ts`:

```
import {
  Controller,
  Get,
  Post,
  Patch,
  Delete,
  Param,
  Body,
  Query,
  ParseUUIDPipe,
  HttpCode,
  HttpStatus,
} from '@nestjs/common';
import { ApiTags, ApiBearerAuth, ApiOperation, ApiResponse } from '@nestjs/swagger';

import { RequirePermission } from '@/core/auth/decorators/permissions.decorator';
import { CurrentUser } from '@/core/auth/decorators/current-user.decorator';
import { CurrentTenant } from '@/core/auth/decorators/current-tenant.decorator';
import { AuthenticatedUser } from '@/core/auth/interfaces/authenticated-
user.interface';
import { Audit } from '@/core/audit/audit.decorator';

import { CustomersService } from './customers.service';
import { CreateCustomerDto } from './dto/create-customer.dto';
import { UpdateCustomerDto } from './dto/update-customer.dto';
import { QueryCustomerDto } from './dto/query-customer.dto';

@ApiTags('Clientes')
@ApiBearerAuth()
@Controller('customers')
export class CustomersController {
  constructor(private readonly customersService: CustomersService) {}

  @Post()
  @RequirePermission('clients:create')
  @Audit('customer.created')
  @ApiOperation({ summary: 'Crear un nuevo cliente' })
  async create(
    @CurrentTenant() tenantId: string,
    @CurrentUser() user: AuthenticatedUser,
    @Body() dto: CreateCustomerDto,
  ) {
    return this.customersService.create(tenantId, dto, user.id);
  }

  @Get()
  @RequirePermission('clients:read')
  @ApiOperation({ summary: 'Listar clientes con filtros y paginación' })
  async findAll(
    @CurrentTenant() tenantId: string,
    @Query() query: QueryCustomerDto,
  ) {
    return this.customersService.findAll(tenantId, query);
  }

  @Get(':id')
  @RequirePermission('clients:read')
  @ApiOperation({ summary: 'Obtener detalle de un cliente' })
  async findOne(
    @CurrentTenant() tenantId: string,
    @Param('id', ParseUUIDPipe) id: string,
  ) {
```

```
    return this.customersService.findOne(tenantId, id);
  }

  @Patch(':id')
  @RequirePermission('clients:update')
  @Audit('customer.updated')
  @ApiOperation({ summary: 'Actualizar un cliente' })
  async update(
    @CurrentTenant() tenantId: string,
    @CurrentUser() user: AuthenticatedUser,
    @Param('id', ParseUUIDPipe) id: string,
    @Body() dto: UpdateCustomerDto,
  ) {
    return this.customersService.update(tenantId, id, dto, user.id);
  }

  @Delete(':id')
  @RequirePermission('clients:delete')
  @Audit('customer.deleted')
  @HttpCode(HttpStatus.NO_CONTENT)
  @ApiOperation({ summary: 'Eliminar un cliente (soft delete)' })
  async remove(
    @CurrentTenant() tenantId: string,
    @Param('id', ParseUUIDPipe) id: string,
  ) {
    return this.customersService.softDelete(tenantId, id);
  }

  @Get(':id/history')
  @RequirePermission('clients:read')
  @ApiOperation({ summary: 'Obtener historial completo del cliente' })
  async getHistory(
    @CurrentTenant() tenantId: string,
    @Param('id', ParseUUIDPipe) id: string,
  ) {
    return this.customersService.getHistory(tenantId, id);
  }
}
```

`customers.service.ts`:

```
import { Injectable, NotFoundException, Logger } from '@nestjs/common';
import { EventEmitter2 } from '@nestjs/event-emitter';

import { PrismaService } from '@/core/database/prisma.service';
import { PaginationMeta } from '@tallerhub/shared';

import { CreateCustomerDto } from './dto/create-customer.dto';
import { UpdateCustomerDto } from './dto/update-customer.dto';
import { QueryCustomerDto } from './dto/query-customer.dto';

@Injectable()
export class CustomersService {
  private readonly logger = new Logger(CustomersService.name);

  constructor(
    private readonly prisma: PrismaService,
    private readonly events: EventEmitter2,
  ) {}

  async create(tenantId: string, dto: CreateCustomerDto, userId: string) {
    const customer = await this.prisma.customer.create({
      data: {
        tenantId,
        ...dto,
        createdBy: userId,
        updatedBy: userId,
      },
    });

    this.events.emit('customer.created', { tenantId, customer });
    return customer;
  }

  async findAll(tenantId: string, query: QueryCustomerDto) {
    const { page = 1, limit = 20, search, status, tags, sortBy, sortDir } = query;
    const skip = (page - 1) * limit;

    const where: any = {
      tenantId,
      deletedAt: null,
    };

    // Filtro de búsqueda full-text
    if (search) {
      where.OR = [
        { firstName: { contains: search, mode: 'insensitive' } },
        { lastName: { contains: search, mode: 'insensitive' } },
        { legalName: { contains: search, mode: 'insensitive' } },
        { email: { contains: search, mode: 'insensitive' } },
        { phone: { contains: search } },
        { taxId: { contains: search } },
      ];
    }

    if (status !== undefined) {
      where.isActive = status === 'active';
    }

    if (tags?.length) {
      where.tags = { hasSome: tags };
    }
```

```
    const [items, total] = await this.prisma.$transaction([
      this.prisma.customer.findMany({
        where,
        skip,
        take: limit,
        orderBy: { [sortBy || 'createdAt']: sortDir || 'desc' },
        include: {
          _count: { select: { workOrders: true, quotes: true, assets: true } },
        },
      }),
      this.prisma.customer.count({ where }),
    ]);

    const meta: PaginationMeta = {
      total,
      page,
      limit,
      totalPages: Math.ceil(total / limit),
      hasNextPage: page < Math.ceil(total / limit),
      hasPrevPage: page > 1,
    };

    return { items, meta };
  }

  async findOne(tenantId: string, id: string) {
    const customer = await this.prisma.customer.findFirst({
      where: { id, tenantId, deletedAt: null },
      include: {
        contacts: true,
        assets: { where: { deletedAt: null }, take: 10 },
        _count: { select: { workOrders: true, quotes: true, assets: true } },
      },
    });

    if (!customer) {
      throw new NotFoundException('Cliente no encontrado');
    }

    return customer;
  }

  async update(tenantId: string, id: string, dto: UpdateCustomerDto, userId: string) {
    await this.findOne(tenantId, id); // Valida existencia y tenant

    const updated = await this.prisma.customer.update({
      where: { id },
      data: { ...dto, updatedBy: userId },
    });

    this.events.emit('customer.updated', { tenantId, customer: updated });
    return updated;
  }

  async softDelete(tenantId: string, id: string) {
    await this.findOne(tenantId, id);

    await this.prisma.customer.update({
      where: { id },
      data: { deletedAt: new Date() },
    });
  }
```

```
  async getHistory(tenantId: string, id: string) {
    await this.findOne(tenantId, id);

    const [workOrders, quotes, auditLogs] = await this.prisma.$transaction([
      this.prisma.workOrder.findMany({
        where: { tenantId, customerId: id, deletedAt: null },
        orderBy: { createdAt: 'desc' },
        take: 50,
        select: {
          id: true, orderNumber: true, status: true,
          totalAmount: true, createdAt: true, completedAt: true,
        },
      }),
      this.prisma.quote.findMany({
        where: { tenantId, customerId: id, deletedAt: null },
        orderBy: { createdAt: 'desc' },
        take: 50,
        select: {
          id: true, quoteNumber: true, status: true,
          total: true, createdAt: true,
        },
      }),
      this.prisma.auditLog.findMany({
        where: { tenantId, entityType: 'customer', entityId: id },
        orderBy: { createdAt: 'desc' },
        take: 100,
      }),
    ]);

    return { workOrders, quotes, auditLogs };
  }
}
```

6.2 Frontend: Ejemplo Completo de Feature — Work Orders

```
src/components/features/work-orders/
├── work-order-form.tsx              # Formulario de creación/edición
├── work-order-table.tsx             # Tabla con filtros
├── work-order-kanban.tsx            # Vista Kanban por estados
├── work-order-detail.tsx            # Vista detallada
├── work-order-products-tab.tsx      # Tab de repuestos
├── work-order-services-tab.tsx      # Tab de servicios
├── work-order-photos-tab.tsx        # Tab de fotos
├── work-order-comments.tsx          # Comentarios
├── work-order-signatures.tsx        # Firmas
├── work-order-checklist.tsx         # Checklist
├── work-order-timeline.tsx          # Timeline de actividad
└── work-order-pdf-button.tsx        # Botón de generación PDF
```

`work-order-table.tsx` **(Client Component):**

```
'use client';

import { useState, useCallback } from 'react';
import { useRouter, useSearchParams } from 'next/navigation';
import { ColumnDef } from '@tanstack/react-table';
import { MoreHorizontal, Plus, Eye, Edit, FileText } from 'lucide-react';

import { Button } from '@/components/ui/button';
import { Badge } from '@/components/ui/badge';
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from '@/components/ui/dropdown-menu';
import { DataTable } from '@/components/shared/data-table/data-table';
import { StatusBadge } from '@/components/shared/status-badge';
import { PriorityBadge } from '@/components/shared/priority-badge';

import { usePermissions } from '@/hooks/use-permissions';
import { formatCurrency, formatDate } from '@/lib/format';
import { WorkOrder } from '@tallerhub/shared';

interface Props {
  initialData: WorkOrder[];
  initialMeta: any;
}

export function WorkOrderTable({ initialData, initialMeta }: Props) {
  const router = useRouter();
  const { can } = usePermissions();

  const columns: ColumnDef<WorkOrder>[] = [
    {
      accessorKey: 'orderNumber',
      header: 'N° Orden',
      cell: ({ row }) => (
        <button
          className="font-mono font-medium text-primary hover:underline"
          onClick={() => router.push(`/work-orders/${row.original.id}`)}
        >
          {row.original.orderNumber}
        </button>
      ),
    },
    {
      accessorKey: 'customer',
      header: 'Cliente',
      cell: ({ row }) => (
        <div>
          <p className="font-medium">
            {row.original.customer?.firstName} {row.original.customer?.lastName}
          </p>
          <p className="text-sm text-muted-foreground">
            {row.original.asset?.brand} {row.original.asset?.model}
          </p>
        </div>
      ),
    },
    {
      accessorKey: 'status',
      header: 'Estado',
```

```
      cell: ({ row }) => <StatusBadge status={row.original.status} />,
    },
    {
      accessorKey: 'priority',
      header: 'Prioridad',
      cell: ({ row }) => <PriorityBadge priority={row.original.priority} />,
    },
    {
      accessorKey: 'assignedTo',
      header: 'Técnico',
      cell: ({ row }) =>
        row.original.assignedUser?.firstName
          ? `${row.original.assignedUser.firstName} ${row.original.assignedUser.lastNa
me}`
          : '—',
    },
    {
      accessorKey: 'totalAmount',
      header: 'Total',
      cell: ({ row }) =>
        can('work_orders:view_costs')
          ? formatCurrency(row.original.totalAmount)
          : '—',
    },
    {
      accessorKey: 'createdAt',
      header: 'Ingreso',
      cell: ({ row }) => formatDate(row.original.createdAt),
    },
    {
      id: 'actions',
      cell: ({ row }) => (
        <DropdownMenu>
          <DropdownMenuTrigger asChild>
            <Button variant="ghost" size="icon">
              <MoreHorizontal className="h-4 w-4" />
            </Button>
          </DropdownMenuTrigger>
          <DropdownMenuContent align="end">
            <DropdownMenuItem onClick={() => router.push(`/work-orders/${row.original.
id}`)}>
              <Eye className="mr-2 h-4 w-4" /> Ver detalle
            </DropdownMenuItem>
            {can('work_orders:update') && (
              <DropdownMenuItem onClick={() => router.push(`/work-orders/${row.origina
l.id}/edit`)}>
                <Edit className="mr-2 h-4 w-4" /> Editar
              </DropdownMenuItem>
            )}
            {can('work_orders:print') && (
              <DropdownMenuItem>
                <FileText className="mr-2 h-4 w-4" /> Generar PDF
              </DropdownMenuItem>
            )}
          </DropdownMenuContent>
        </DropdownMenu>
      ),
    },
  ];

  return (
    <DataTable
      columns={columns}
```

```
      data={initialData}
      meta={initialMeta
}
      searchPlaceholder="Buscar órdenes..."
      filterableColumns={[
        { id: 'status', title: 'Estado', options: STATUS_OPTIONS },
        { id: 'priority', title: 'Prioridad', options: PRIORITY_OPTIONS },
      ]}
      onRowClick={(row) => router.push(`/work-orders/${row.id}`)}
    />
  );
}
```

6.3 Ejemplo Completo: Pantalla Móvil

```
apps/mobile/src/features/work-orders/
├── screens/
│   ├── WorkOrderListScreen.tsx
│   ├── WorkOrderDetailScreen.tsx
│   └── WorkOrderFormScreen.tsx
├── components/
│   ├── WorkOrderCard.tsx
│   ├── StatusBadge.tsx
│   └── VehicleInfo.tsx
├── hooks/
│   ├── useWorkOrders.ts
│   └── useWorkOrderSync.ts
└── navigation/
    └── WorkOrderStack.tsx
```

`WorkOrderListScreen.tsx`

```
// apps/mobile/src/features/work-orders/screens/WorkOrderListScreen.tsx
import React, { useCallback, useState } from 'react';
import { FlatList, RefreshControl, View } from 'react-native';
import { useNavigation } from '@react-navigation/native';
import { useInfiniteQuery } from '@tanstack/react-query';

import { apiClient } from '@/lib/api-client';
import { useNetworkStatus } from '@/hooks/useNetworkStatus';
import { useOfflineStore } from '@/stores/offline-store';
import { WorkOrderCard } from '../components/WorkOrderCard';
import { EmptyState } from '@/components/EmptyState';
import { ErrorState } from '@/components/ErrorState';
import { FloatingActionButton } from '@/components/FloatingActionButton';
import { SearchBar } from '@/components/SearchBar';

import type { WorkOrder } from '@tallerhub/shared';

export function WorkOrderListScreen() {
  const navigation = useNavigation();
  const { isOnline } = useNetworkStatus();
  const offlineStore = useOfflineStore();
  const [search, setSearch] = useState('');

  const {
    data,
    fetchNextPage,
    hasNextPage,
    isFetchingNextPage,
    isLoading,
    isError,
    error,
    refetch,
  } = useInfiniteQuery({
    queryKey: ['work-orders', search],
    queryFn: ({ pageParam }) =>
      apiClient.get<WorkOrder[]>('/work-orders', {
        params: { cursor: pageParam, search, limit: 20 },
      }),
    getNextPageParam: (lastPage) => lastPage.meta?.nextCursor,
    enabled: isOnline,
  });

  // Combinar datos online con pendientes offline
  const workOrders = React.useMemo(() => {
    const onlineData = data?.pages.flatMap((p) => p.data) ?? [];
    const pendingOffline = offlineStore.getPending('work-orders');
    return [...pendingOffline, ...onlineData];
  }, [data, offlineStore]);

  const handleRefresh = useCallback(async () => {
    if (isOnline) await refetch();
  }, [isOnline, refetch]);

  const renderItem = useCallback(
    ({ item }: { item: WorkOrder }) => (
      <WorkOrderCard
        workOrder={item}
        onPress={() => navigation.navigate('WorkOrderDetail', { id: item.id })}
        isPending={item._isPending}
      />
    ),
    [navigation],
```

```
  );

  if (isError) {
    return <ErrorState message={error.message} onRetry={refetch} />;
  }

  return (
    <View className="flex-1 bg-background">
      <SearchBar
        value={search}
        onChangeText={setSearch}
        placeholder="Buscar por número, cliente o vehículo..."
      />

      <FlatList
        data={workOrders}
        renderItem={renderItem}
        keyExtractor={(item) => item.id}
        contentContainerStyle={{ padding: 16, gap: 12 }}
        refreshControl={
          <RefreshControl refreshing={isLoading} onRefresh={handleRefresh} />
        }
        onEndReached={() => hasNextPage && fetchNextPage()}
        onEndReachedThreshold={0.5}
        ListEmptyComponent={
          !isLoading ? (
            <EmptyState
              icon="clipboard-list"
              title="Sin órdenes de trabajo"
              description="Crea tu primera orden de trabajo"
            />
          ) : null
        }
        ListFooterComponent={
          isFetchingNextPage ? <ActivityIndicator style={{ padding: 16 }} /> : null
        }
      />

      <FloatingActionButton
        icon="plus"
        onPress={() => navigation.navigate('WorkOrderForm')}
      />
    </View>
  );
}
```

6.4 Ejemplo: Tipos Compartidos (`@tallerhub/shared`)

```
packages/shared/src/
├── types/
│   ├── index.ts
│   ├── work-order.types.ts
│   ├── customer.types.ts
│   ├── vehicle.types.ts
│   ├── product.types.ts
│   └── api.types.ts
├── enums/
│   ├── index.ts
│   ├── work-order-status.enum.ts
│   └── user-role.enum.ts
├── validators/
│   ├── index.ts
│   ├── work-order.validators.ts
│   └── customer.validators.ts
├── constants/
│   ├── index.ts
│   ├── permissions.ts
│   └── status-transitions.ts
└── utils/
    ├── index.ts
    ├── currency.ts
    ├── date.ts
    └── slug.ts
```

`types/work-order.types.ts`

```typescript
// packages/shared/src/types/work-order.types.ts

export interface WorkOrder {
  id: string;
  orderNumber: string;
  tenantId: string;
  branchId: string;
  customerId: string;
  vehicleId: string;
  assignedToId: string | null;
  status: WorkOrderStatus;
  priority: WorkOrderPriority;
  diagnosticNotes: string | null;
  internalNotes: string | null;
  estimatedCompletionDate: Date | null;
  completedAt: Date | null;
  subtotal: number;
  taxAmount: number;
  discountAmount: number;
  totalAmount: number;
  createdAt: Date;
  updatedAt: Date;

  // Relaciones opcionales (populated)
  customer?: Customer;
  vehicle?: Vehicle;
  assignedTo?: User;
  branch?: Branch;
  items?: WorkOrderItem[];
}

export interface WorkOrderItem {
  id: string;
  workOrderId: string;
  type: 'SERVICE' | 'PRODUCT';
  description: string;
  quantity: number;
  unitPrice: number;
  discount: number;
  totalPrice: number;
  serviceId?: string;
  productId?: string;
  technicianId?: string;
}

export interface CreateWorkOrderDto {
  customerId: string;
  vehicleId: string;
  branchId: string;
  assignedToId?: string;
  priority?: WorkOrderPriority;
  diagnosticNotes?: string;
  estimatedCompletionDate?: string; // ISO 8601
  items: CreateWorkOrderItemDto[];
}

export interface CreateWorkOrderItemDto {
  type: 'SERVICE' | 'PRODUCT';
  description: string;
  quantity: number;
  unitPrice: number;
  discount?: number;
```

```
  serviceId?: string;
  productId?: string;
  technicianId?: string;
}
```

`enums/work-order-status.enum.ts`

```
// packages/shared/src/enums/work-order-status.enum.ts

export enum WorkOrderStatus {
  PENDING = 'PENDING',
  DIAGNOSED = 'DIAGNOSED',
  QUOTED = 'QUOTED',
  APPROVED = 'APPROVED',
  IN_PROGRESS = 'IN_PROGRESS',
  WAITING_PARTS = 'WAITING_PARTS',
  QUALITY_CHECK = 'QUALITY_CHECK',
  COMPLETED = 'COMPLETED',
  DELIVERED = 'DELIVERED',
  CANCELLED = 'CANCELLED',
}

export enum WorkOrderPriority {
  LOW = 'LOW',
  MEDIUM = 'MEDIUM',
  HIGH = 'HIGH',
  URGENT = 'URGENT',
}

/**
 * Transiciones de estado válidas.
 * Clave = estado actual, valor = estados posibles siguientes.
 */
export const WORK_ORDER_TRANSITIONS: Record<WorkOrderStatus, WorkOrderStatus[]> = {
  [WorkOrderStatus.PENDING]: [WorkOrderStatus.DIAGNOSED, WorkOrderStatus.CANCELLED],
  [WorkOrderStatus.DIAGNOSED]: [WorkOrderStatus.QUOTED, WorkOrderStatus.CANCELLED],
  [WorkOrderStatus.QUOTED]: [WorkOrderStatus.APPROVED, WorkOrderStatus.CANCELLED],
  [WorkOrderStatus.APPROVED]: [WorkOrderStatus.IN_PROGRESS, WorkOrder-
Status.CANCELLED],
  [WorkOrderStatus.IN_PROGRESS]: [
    WorkOrderStatus.WAITING_PARTS,
    WorkOrderStatus.QUALITY_CHECK,
    WorkOrderStatus.CANCELLED,
  ],
  [WorkOrderStatus.WAITING_PARTS]: [WorkOrderStatus.IN_PROGRESS],
  [WorkOrderStatus.QUALITY_CHECK]: [WorkOrderStatus.COMPLETED, WorkOrder-
Status.IN_PROGRESS],
  [WorkOrderStatus.COMPLETED]: [WorkOrderStatus.DELIVERED],
  [WorkOrderStatus.DELIVERED]: [],
  [WorkOrderStatus.CANCELLED]: [],
};

/**
 * Valida si una transición de estado es permitida.
 */
export function isValidTransition(
  currentStatus: WorkOrderStatus,
  newStatus: WorkOrderStatus,
): boolean {
  return WORK_ORDER_TRANSITIONS[currentStatus]?.includes(newStatus) ?? false;
}
```

`validators/work-order.validators.ts`

```
// packages/shared/src/validators/work-order.validators.ts
import { z } from 'zod';
import { WorkOrderPriority } from '../enums/work-order-status.enum';

export const createWorkOrderItemSchema = z.object({
  type: z.enum(['SERVICE', 'PRODUCT']),
  description: z.string().min(1).max(500),
  quantity: z.number().positive().max(9999),
  unitPrice: z.number().nonnegative().max(99_999_999),
  discount: z.number().min(0).max(100).default(0),
  serviceId: z.string().uuid().optional(),
  productId: z.string().uuid().optional(),
  technicianId: z.string().uuid().optional(),
});

export const createWorkOrderSchema = z.object({
  customerId: z.string().uuid(),
  vehicleId: z.string().uuid(),
  branchId: z.string().uuid(),
  assignedToId: z.string().uuid().optional(),
  priority: z.nativeEnum(WorkOrderPriority).default(WorkOrderPriority.MEDIUM),
  diagnosticNotes: z.string().max(5000).optional(),
  estimatedCompletionDate: z.string().datetime().optional(),
  items: z.array(createWorkOrderItemSchema).min(1, 'Debe incluir al menos un ítem'),
});

export type CreateWorkOrderInput = z.infer<typeof createWorkOrderSchema>;
export type CreateWorkOrderItemInput = z.infer<typeof createWorkOrderItemSchema>;
```

---

7. Estrategia Multi-Tenant en Código

7.1 Visión General

TallerHub utiliza **Shared Schema con Discriminador** `tenant_id` y `AsyncLocalStorage` para propagar el contexto de tenant de forma transparente a través de toda la solicitud HTTP.

```
┌──────────────────────────────────────────────────────────────────┐
│                       Flujo de Request                           │
│                                                                  │
│  Request → TenantMiddleware → AsyncLocalStorage.run()            │
│              │                        │                          │
│              ▼                        ▼                          │
│         Extrae tenant_id       Guarda contexto                   │
│         del JWT/header         { tenantId, userId, ... }         │
│              │                        │                          │
│              ▼                        ▼                          │
│         TenantGuard            PrismaMiddleware                  │
│         (valida acceso)        (inyecta WHERE tenant_id)         │
│              │                        │                          │
│              ▼                        ▼                          │
│         Controller             Queries filtradas                 │
│         (lógica de negocio)    automáticamente                   │
└──────────────────────────────────────────────────────────────────┘
```

7.2 Implementación de `AsyncLocalStorage` Context

```
// apps/backend/src/common/tenant/tenant-context.ts
import { AsyncLocalStorage } from 'async_hooks';

export interface TenantContext {
  tenantId: string;
  userId: string;
  branchId: string | null;
  roles: string[];
}

export const tenantStorage = new AsyncLocalStorage<TenantContext>();

/**
 * Obtener el contexto de tenant actual.
 * Lanza error si se llama fuera de un contexto de request.
 */
export function getCurrentTenant(): TenantContext {
  const context = tenantStorage.getStore();
  if (!context) {
    throw new Error('Tenant context not available. Ensure TenantMiddleware is ap-
plied.');
  }
  return context;
}

/**
 * Obtener tenantId del contexto actual (shorthand).
 */
export function getCurrentTenantId(): string {
  return getCurrentTenant().tenantId;
}
```

7.3 Tenant Middleware

```
// apps/backend/src/common/tenant/tenant.middleware.ts
import { Injectable, NestMiddleware, UnauthorizedException } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';
import { JwtService } from '@nestjs/jwt';
import { tenantStorage, TenantContext } from './tenant-context';
import { PrismaService } from '@/prisma/prisma.service';

@Injectable()
export class TenantMiddleware implements NestMiddleware {
  constructor(
    private readonly jwt: JwtService,
    private readonly prisma: PrismaService,
  ) {}

  async use(req: Request, _res: Response, next: NextFunction): Promise<void> {
    const token = this.extractToken(req);
    if (!token) {
      // Rutas públicas no requieren tenant
      return next();
    }

    try {
      const payload = this.jwt.verify(token);

      // Validar que el tenant existe y está activo
      const tenant = await this.prisma.tenant.findUnique({
        where: { id: payload.tenantId },
        select: { id: true, status: true, plan: true },
      });

      if (!tenant || tenant.status !== 'ACTIVE') {
        throw new UnauthorizedException('Tenant inactivo o no encontrado');
      }

      const context: TenantContext = {
        tenantId: payload.tenantId,
        userId: payload.sub,
        branchId: payload.branchId ?? null,
        roles: payload.roles ?? [],
      };

      // Ejecutar el resto del request dentro del contexto del tenant
      tenantStorage.run(context, () => {
        // Adjuntar al request para acceso directo en decoradores
        (req as any).tenantContext = context;
        next();
      });
    } catch (error) {
      if (error instanceof UnauthorizedException) throw error;
      throw new UnauthorizedException('Token inválido');
    }
  }

  private extractToken(req: Request): string | null {
    const auth = req.headers.authorization;
    if (!auth?.startsWith('Bearer ')) return null;
    return auth.slice(7);
  }
}
```

7.4 Decorador `@CurrentTenant`

```
// apps/backend/src/common/tenant/current-tenant.decorator.ts
import { createParamDecorator, ExecutionContext } from '@nestjs/common';
import { TenantContext } from './tenant-context';

export const CurrentTenant = createParamDecorator(
  (data: keyof TenantContext | undefined, ctx: ExecutionContext) => {
    const request = ctx.switchToHttp().getRequest();
    const tenantContext: TenantContext = request.tenantContext;

    if (!tenantContext) {
      throw new Error('TenantContext not found. Is TenantMiddleware applied?');
    }

    return data ? tenantContext[data] : tenantContext;
  },
);

// Uso en controladores:
// @CurrentTenant() tenant: TenantContext
// @CurrentTenant('tenantId') tenantId: string
// @CurrentTenant('branchId') branchId: string | null
```

7.5 Tenant Guard

```
// apps/backend/src/common/tenant/tenant.guard.ts
import {
  CanActivate,
  ExecutionContext,
  ForbiddenException,
  Injectable,
} from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { IS_PUBLIC_KEY } from '../auth/public.decorator';

@Injectable()
export class TenantGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(ctx: ExecutionContext): boolean {
    // Saltar validación en rutas públicas
    const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
      ctx.getHandler(),
      ctx.getClass(),
    ]);
    if (isPublic) return true;

    const request = ctx.switchToHttp().getRequest();
    const tenantContext = request.tenantContext;

    if (!tenantContext?.tenantId) {
      throw new ForbiddenException('Contexto de tenant requerido');
    }

    // Validar que el recurso solicitado pertenece al tenant
    const resourceTenantId = request.params?.tenantId || request.body?.tenantId;
    if (resourceTenantId && resourceTenantId !== tenantContext.tenantId) {
      throw new ForbiddenException('Acceso denegado: recurso de otro tenant');
    }

    return true;
  }
}
```

7.6 Prisma Middleware para Inyección Automática de `tenant_id`

```
// apps/backend/src/prisma/prisma-tenant.middleware.ts
import { Prisma } from '@prisma/client';
import { tenantStorage } from '@/common/tenant/tenant-context';

/**
 * Modelos que NO requieren filtrado por tenant (tablas globales).
 */
const GLOBAL_MODELS = new Set([
  'Tenant',
  'Plan',
  'SystemConfig',
  'AuditLog',
]);

/**
 * Middleware de Prisma que automáticamente:
 * 1. Inyecta `tenantId` en CREATE
 * 2. Añade filtro `tenantId` en READ/UPDATE/DELETE
 */
export function tenantPrismaMiddleware(): Prisma.Middleware {
  return async (params: Prisma.MiddlewareParams, next) => {
    const context = tenantStorage.getStore();

    // Si no hay contexto de tenant o es modelo global, continuar sin modificar
    if (!context?.tenantId || GLOBAL_MODELS.has(params.model ?? '')) {
      return next(params);
    }

    const tenantId = context.tenantId;

    switch (params.action) {
      // ─── CREATE ─────────────────────────────────────────
      case 'create':
        params.args.data = { ...params.args.data, tenantId };
        break;

      case 'createMany':
        if (Array.isArray(params.args.data)) {
          params.args.data = params.args.data.map((d: any) => ({
            ...d,
            tenantId,
          }));
        } else {
          params.args.data = { ...params.args.data, tenantId };
        }
        break;

      // ─── READ ───────────────────────────────────────────
      case 'findUnique':
      case 'findFirst':
        params.args.where = { ...params.args.where, tenantId };
        break;

      case 'findMany':
      case 'count':
      case 'aggregate':
      case 'groupBy':
        params.args.where = { ...params.args.where, tenantId };
        break;

      // ─── UPDATE ─────────────────────────────────────────
      case 'update':
```

```
        params.args.where = { ...params.args.where, tenantId };
        break;

      case 'updateMany':
        params.args.where = { ...params.args.where, tenantId };
        break;

      // ─── DELETE ──────────────────────────────────────────
      case 'delete':
        params.args.where = { ...params.args.where, tenantId };
        break;

      case 'deleteMany':
        params.args.where = { ...params.args.where, tenantId };
        break;

      // ─── UPSERT ──────────────────────────────────────────
      case 'upsert':
        params.args.where = { ...params.args.where, tenantId };
        params.args.create = { ...params.args.create, tenantId };
        break;
    }

    return next(params);
  };
}
```

7.7 Registro del Middleware en Prisma Service

```
// apps/backend/src/prisma/prisma.service.ts (actualizado)
import { Injectable, OnModuleInit, OnModuleDestroy, Logger } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';
import { tenantPrismaMiddleware } from './prisma-tenant.middleware';

@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit, OnModuleDes-
troy {
  private readonly logger = new Logger(PrismaService.name);

  constructor() {
    super({
      log: [
        { emit: 'event', level: 'query' },
        { emit: 'event', level: 'error' },
        { emit: 'event', level: 'warn' },
      ],
    });

    // Registrar middleware de tenant
    this.$use(tenantPrismaMiddleware());
  }

  async onModuleInit() {
    await this.$connect();
    this.logger.log('Prisma conectado a la base de datos');

    // Log de queries lentas en desarrollo
    if (process.env.NODE_ENV === 'development') {
      (this as any).$on('query', (e: any) => {
        if (e.duration > 500) {
          this.logger.warn(`Query lenta (${e.duration}ms): ${e.query}`);
        }
      });
    }
  }

  async onModuleDestroy() {
    await this.$disconnect();
  }

  /**
   * Helper para transacciones con retry automático.
   */
  async executeInTransaction<T>(
    fn: (tx: Prisma.TransactionClient) => Promise<T>,
    maxRetries = 3,
  ): Promise<T> {
    let retries = 0;
    while (true) {
      try {
        return await this.$transaction(fn, {
          maxWait: 5000,
          timeout: 10000,
          isolationLevel: 'ReadCommitted',
        });
      } catch (error) {
        if (
          error instanceof Prisma.PrismaClientKnownRequestError &&
          error.code === 'P2034' &&
          retries < maxRetries
        ) {
```

```
          retries++;
          this.logger.warn(`Transaction conflict, retry ${retries}/${maxRetries}`);
          await new Promise((r) => setTimeout(r, 100 * retries));
          continue;
        }
        throw error;
      }
    }
  }
}
```

7.8 Testing Multi-Tenant

```
// apps/backend/test/tenant/tenant-isolation.e2e-spec.ts
import { Test } from '@nestjs/testing';
import { INestApplication } from '@nestjs/common';
import * as request from 'supertest';
import { AppModule } from '@/app.module';
import { PrismaService } from '@/prisma/prisma.service';

describe('Aislamiento Multi-Tenant (E2E)', () => {
  let app: INestApplication;
  let prisma: PrismaService;
  let tokenTenantA: string;
  let tokenTenantB: string;

  beforeAll(async () => {
    const module = await Test.createTestingModule({
      imports: [AppModule],
    }).compile();

    app = module.createNestApplication();
    prisma = module.get(PrismaService);
    await app.init();

    // Crear dos tenants de prueba con usuarios
    // (setup omitido por brevedad — ver test helpers)
    tokenTenantA = await getAuthToken(app, 'admin@tenant-a.com');
    tokenTenantB = await getAuthToken(app, 'admin@tenant-b.com');
  });

  afterAll(async () => {
    await prisma.$disconnect();
    await app.close();
  });

  it('Tenant A no puede ver clientes de Tenant B', async () => {
    // Crear cliente en Tenant B
    const { body: created } = await request(app.getHttpServer())
      .post('/api/v1/customers')
      .set('Authorization', `Bearer ${tokenTenantB}`)
      .send({
        firstName: 'Carlos',
        lastName: 'López',
        email: 'carlos@tenantb.com',
        phone: '+56912345678',
      })
      .expect(201);

    // Intentar acceder desde Tenant A
    await request(app.getHttpServer())
      .get(`/api/v1/customers/${created.data.id}`)
      .set('Authorization', `Bearer ${tokenTenantA}`)
      .expect(404); // No encontrado (filtrado por tenant)
  });

  it('Tenant A no puede actualizar recursos de Tenant B', async () => {
    const customerB = await prisma.customer.findFirst({
      where: { tenant: { slug: 'tenant-b' } },
    });

    await request(app.getHttpServer())
      .patch(`/api/v1/customers/${customerB!.id}`)
      .set('Authorization', `Bearer ${tokenTenantA}`)
      .send({ firstName: 'Hackeado' })
```

```
      .expect(404);

    // Verificar que no se modificó
    const unchanged = await prisma.customer.findUnique({
      where: { id: customerB!.id },
    });
    expect(unchanged!.firstName).not.toBe('Hackeado');
  });

  it('Las listas solo muestran datos del tenant autenticado', async () => {
    const { body } = await request(app.getHttpServer())
      .get('/api/v1/customers')
      .set('Authorization', `Bearer ${tokenTenantA}`)
      .expect(200);

    // Verificar que todos los resultados pertenecen a Tenant A
    const tenantA = await prisma.tenant.findFirst({ where: { slug: 'tenant-a' } });
    for (const customer of body.data) {
      expect(customer.tenantId).toBe(tenantA!.id);
    }
  });

  it('El conteo respeta el aislamiento de tenant', async () => {
    const { body: countA } = await request(app.getHttpServer())
      .get('/api/v1/customers?countOnly=true')
      .set('Authorization', `Bearer ${tokenTenantA}`)
      .expect(200);

    const { body: countB } = await request(app.getHttpServer())
      .get('/api/v1/customers?countOnly=true')
      .set('Authorization', `Bearer ${tokenTenantB}`)
      .expect(200);

    // Los conteos deben ser independientes
    const dbCountA = await prisma.customer.count({
      where: { tenantId: (await prisma.tenant.findFirst({ where: { slug: 'tenant-
a' } }))!.id },
    });

    expect(countA.data.total).toBe(dbCountA);
    expect(countA.data.total).not.toBe(countB.data.total);
  });
});
```

---

8. Estrategia RBAC en Código

8.1 Visión General

El sistema RBAC de TallerHub implementa control de acceso basado en **permisos granulares** asignados a **roles**, con soporte para **permisos por sucursal** y **caché en Redis**.

```
┌──────────────────────────────────────────────────────────────────────┐
│                      Flujo de Autorización                           │
│                                                                      │
│  @Permissions('work_orders:update')                                  │
│       │                                                              │
│       ▼                                                              │
│  PermissionsGuard.canActivate()                                      │
│       │                                                              │
│       ├──→ Redis Cache Hit? ──→ Sí ──→ Verificar permiso             │
│       │                                                              │
│       └──→ No ──→ PermissionsService.getUserPermissions()            │
│                        │                                             │
│                        ▼                                             │
│                 DB Query (user → roles → permissions)                │
│                        │                                             │
│                        ▼                                             │
│                 Guardar en Redis (TTL: 5 min)                        │
│                        │                                             │
│                        ▼                                             │
│                 Verificar permiso ──→ Allow / Deny                   │
└──────────────────────────────────────────────────────────────────────┘
```

8.2 Constantes de Permisos

```
// packages/shared/src/constants/permissions.ts

/**
 * Mapa completo de permisos del sistema.
 * Formato: módulo:acción
 */
export const PERMISSIONS = {
  // ─── Clientes ──────────────────────────────────
  CUSTOMERS_CREATE: 'customers:create',
  CUSTOMERS_READ: 'customers:read',
  CUSTOMERS_UPDATE: 'customers:update',
  CUSTOMERS_DELETE: 'customers:delete',
  CUSTOMERS_EXPORT: 'customers:export',

  // ─── Órdenes de Trabajo ────────────────────────
  WORK_ORDERS_CREATE: 'work_orders:create',
  WORK_ORDERS_READ: 'work_orders:read',
  WORK_ORDERS_UPDATE: 'work_orders:update',
  WORK_ORDERS_DELETE: 'work_orders:delete',
  WORK_ORDERS_ASSIGN: 'work_orders:assign',
  WORK_ORDERS_CHANGE_STATUS: 'work_orders:change_status',
  WORK_ORDERS_PRINT: 'work_orders:print',

  // ─── Cotizaciones ──────────────────────────────
  QUOTES_CREATE: 'quotes:create',
  QUOTES_READ: 'quotes:read',
  QUOTES_UPDATE: 'quotes:update',
  QUOTES_DELETE: 'quotes:delete',
  QUOTES_APPROVE: 'quotes:approve',
  QUOTES_SEND: 'quotes:send',

  // ─── Inventario ────────────────────────────────
  INVENTORY_READ: 'inventory:read',
  INVENTORY_ADJUST: 'inventory:adjust',
  INVENTORY_TRANSFER: 'inventory:transfer',
  INVENTORY_COUNT: 'inventory:count',

  // ─── Productos ─────────────────────────────────
  PRODUCTS_CREATE: 'products:create',
  PRODUCTS_READ: 'products:read',
  PRODUCTS_UPDATE: 'products:update',
  PRODUCTS_DELETE: 'products:delete',
  PRODUCTS_IMPORT: 'products:import',

  // ─── Servicios ─────────────────────────────────
  SERVICES_CREATE: 'services:create',
  SERVICES_READ: 'services:read',
  SERVICES_UPDATE: 'services:update',
  SERVICES_DELETE: 'services:delete',

  // ─── Reportes ──────────────────────────────────
  REPORTS_SALES: 'reports:sales',
  REPORTS_INVENTORY: 'reports:inventory',
  REPORTS_PRODUCTIVITY: 'reports:productivity',
  REPORTS_FINANCIAL: 'reports:financial',
  REPORTS_EXPORT: 'reports:export',

  // ─── Administración ────────────────────────────
  ADMIN_USERS: 'admin:users',
  ADMIN_ROLES: 'admin:roles',
  ADMIN_BRANCHES: 'admin:branches',
  ADMIN_SETTINGS: 'admin:settings',
```

```
  ADMIN_BILLING: 'admin:billing',
  ADMIN_AUDIT_LOG: 'admin:audit_log',
} as const;

export type Permission = (typeof PERMISSIONS)[keyof typeof PERMISSIONS];
```

8.3 Decorador `@Permissions`

```
// apps/backend/src/common/auth/permissions.decorator.ts
import { SetMetadata } from '@nestjs/common';
import { Permission } from '@tallerhub/shared';

export const PERMISSIONS_KEY = 'permissions';

/**
 * Decorador que marca los permisos requeridos para un endpoint.
 * Se pueden pasar múltiples permisos (se requiere AL MENOS UNO).
 *
 * @example
 * @Permissions('work_orders:create')
 * @Permissions('work_orders:update', 'work_orders:assign')
 */
export const Permissions = (...permissions: Permission[]) =>
  SetMetadata(PERMISSIONS_KEY, permissions);
```

8.4 Permissions Guard

```
// apps/backend/src/common/auth/permissions.guard.ts
import {
  CanActivate,
  ExecutionContext,
  ForbiddenException,
  Injectable,
  Logger,
} from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { PERMISSIONS_KEY } from './permissions.decorator';
import { IS_PUBLIC_KEY } from './public.decorator';
import { PermissionsService } from './permissions.service';
import { TenantContext } from '../tenant/tenant-context';
import type { Permission } from '@tallerhub/shared';

@Injectable()
export class PermissionsGuard implements CanActivate {
  private readonly logger = new Logger(PermissionsGuard.name);

  constructor(
    private readonly reflector: Reflector,
    private readonly permissionsService: PermissionsService,
  ) {}

  async canActivate(ctx: ExecutionContext): Promise<boolean> {
    // 1. Saltar en rutas públicas
    const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
      ctx.getHandler(),
      ctx.getClass(),
    ]);
    if (isPublic) return true;

    // 2. Obtener permisos requeridos del decorador
    const requiredPermissions = this.reflector.getAllAndOverride<Permission[]>(
      PERMISSIONS_KEY,
      [ctx.getHandler(), ctx.getClass()],
    );

    // Si no se especifican permisos, solo requiere autenticación
    if (!requiredPermissions?.length) return true;

    // 3. Obtener contexto del tenant
    const request = ctx.switchToHttp().getRequest();
    const tenantContext: TenantContext = request.tenantContext;

    if (!tenantContext) {
      throw new ForbiddenException('Contexto de autenticación requerido');
    }

    // 4. Super Admin tiene acceso total
    if (tenantContext.roles.includes('SUPER_ADMIN')) return true;

    // 5. Obtener permisos del usuario (con caché Redis)
    const userPermissions = await this.permissionsService.getUserPermissions(
      tenantContext.tenantId,
      tenantContext.userId,
      tenantContext.branchId,
    );

    // 6. Verificar que tiene AL MENOS UNO de los permisos requeridos
    const hasPermission = requiredPermissions.some((perm) =>
      userPermissions.includes(perm),
```

```
    );

    if (!hasPermission) {
      this.logger.warn(
        `Acceso denegado: usuario ${tenantContext.userId} sin permisos [${requiredPer-
missions.join(', ')}]`,
      );
      throw new ForbiddenException(
        `Permisos insuficientes. Se requiere: ${requiredPermissions.join(' o ')}`,
      );
    }

    return true;
  }
}
```

8.5 Permissions Service con Caché Redis

```
// apps/backend/src/common/auth/permissions.service.ts
import { Injectable, Logger } from '@nestjs/common';
import { InjectRedis } from '@nestjs-modules/ioredis';
import Redis from 'ioredis';
import { PrismaService } from '@/prisma/prisma.service';
import type { Permission } from '@tallerhub/shared';

@Injectable()
export class PermissionsService {
  private readonly logger = new Logger(PermissionsService.name);
  private readonly CACHE_TTL = 300; // 5 minutos

  constructor(
    @InjectRedis() private readonly redis: Redis,
    private readonly prisma: PrismaService,
  ) {}

  /**
   * Obtiene los permisos de un usuario, usando caché Redis.
   */
  async getUserPermissions(
    tenantId: string,
    userId: string,
    branchId: string | null,
  ): Promise<Permission[]> {
    const cacheKey = this.buildCacheKey(tenantId, userId, branchId);

    // 1. Intentar obtener del caché
    const cached = await this.redis.get(cacheKey);
    if (cached) {
      return JSON.parse(cached) as Permission[];
    }

    // 2. Consultar desde la base de datos
    const permissions = await this.fetchPermissionsFromDb(tenantId, userId, branchId);

    // 3. Guardar en caché
    await this.redis.setex(cacheKey, this.CACHE_TTL, JSON.stringify(permissions));

    return permissions;
  }

  /**
   * Invalida el caché de permisos de un usuario.
   * Llamar cuando se actualicen roles/permisos.
   */
  async invalidateUserPermissions(tenantId: string, userId: string): Promise<void> {
    const pattern = `tenant:${tenantId}:user:${userId}:permissions:*`;
    const keys = await this.redis.keys(pattern);

    if (keys.length > 0) {
      await this.redis.del(...keys);
      this.logger.debug(`Cache invalidado: ${keys.length} keys para usuario ${userId}
`);
    }
  }

  /**
   * Invalida todos los cachés de permisos de un tenant.
   * Llamar cuando se modifique la configuración de roles del tenant.
   */
  async invalidateTenantPermissions(tenantId: string): Promise<void> {
```

```
    const pattern = `tenant:${tenantId}:user:*:permissions:*`;
    const keys = await this.redis.keys(pattern);

    if (keys.length > 0) {
      await this.redis.del(...keys);
      this.logger.log(`Cache de tenant ${tenantId} invalidado: ${keys.length} keys`);
    }
  }

  // ─── Privados ──────────────────────────────

  private buildCacheKey(tenantId: string, userId: string, branchId: string | null): st
ring {
    const branch = branchId ?? 'all';
    return `tenant:${tenantId}:user:${userId}:permissions:${branch}`;
  }

  private async fetchPermissionsFromDb(
    tenantId: string,
    userId: string,
    branchId: string | null,
  ): Promise<Permission[]> {
    // Obtener roles del usuario en el tenant (y opcionalmente en la sucursal)
    const userRoles = await this.prisma.userRole.findMany({
      where: {
        userId,
        tenantId,
        ...(branchId ? { OR: [{ branchId }, { branchId: null }] } : {}),
      },
      include: {
        role: {
          include: {
            permissions: {
              include: { permission: true },
            },
          },
        },
      },
    });

    // Combinar permisos de todos los roles (deduplicar)
    const permissionSet = new Set<Permission>();

    for (const userRole of userRoles) {
      for (const rolePermission of userRole.role.permissions) {
        permissionSet.add(rolePermission.permission.code as Permission);
      }
    }

    return Array.from(permissionSet);
  }
}
```

8.6 Hook Frontend: `usePermissions`

```
// apps/web/src/hooks/usePermissions.ts
'use client';

import { useCallback, useMemo } from 'react';
import { useSession } from '@/providers/session-provider';
import type { Permission } from '@tallerhub/shared';

interface UsePermissionsReturn {
  /** Verifica si el usuario tiene el permiso especificado */
  can: (permission: Permission) => boolean;
  /** Verifica si el usuario tiene TODOS los permisos */
  canAll: (...permissions: Permission[]) => boolean;
  /** Verifica si el usuario tiene AL MENOS UNO de los permisos */
  canAny: (...permissions: Permission[]) => boolean;
  /** Verifica si es Super Admin */
  isSuperAdmin: boolean;
  /** Lista de permisos del usuario */
  permissions: Permission[];
}

export function usePermissions(): UsePermissionsReturn {
  const { user } = useSession();

  const permissions = useMemo(
    () => (user?.permissions ?? []) as Permission[],
    [user?.permissions],
  );

  const isSuperAdmin = useMemo(
    () => user?.roles?.includes('SUPER_ADMIN') ?? false,
    [user?.roles],
  );

  const can = useCallback(
    (permission: Permission): boolean => {
      if (isSuperAdmin) return true;
      return permissions.includes(permission);
    },
    [permissions, isSuperAdmin],
  );

  const canAll = useCallback(
    (...required: Permission[]): boolean => {
      if (isSuperAdmin) return true;
      return required.every((p) => permissions.includes(p));
    },
    [permissions, isSuperAdmin],
  );

  const canAny = useCallback(
    (...required: Permission[]): boolean => {
      if (isSuperAdmin) return true;
      return required.some((p) => permissions.includes(p));
    },
    [permissions, isSuperAdmin],
  );

  return { can, canAll, canAny, isSuperAdmin, permissions };
}
```

8.7 Componente de Protección Frontend

```
// apps/web/src/components/auth/PermissionGate.tsx
'use client';

import { usePermissions } from '@/hooks/usePermissions';
import type { Permission } from '@tallerhub/shared';

interface PermissionGateProps {
  /** Permiso requerido */
  permission?: Permission;
  /** Múltiples permisos — requiere al menos uno */
  anyOf?: Permission[];
  /** Múltiples permisos — requiere todos */
  allOf?: Permission[];
  /** Contenido a renderizar si tiene permiso */
  children: React.ReactNode;
  /** Contenido alternativo si no tiene permiso */
  fallback?: React.ReactNode;
}

export function PermissionGate({
  permission,
  anyOf,
  allOf,
  children,
  fallback = null,
}: PermissionGateProps) {
  const { can, canAny, canAll } = usePermissions();

  let hasAccess = false;

  if (permission) {
    hasAccess = can(permission);
  } else if (anyOf?.length) {
    hasAccess = canAny(...anyOf);
  } else if (allOf?.length) {
    hasAccess = canAll(...allOf);
  } else {
    hasAccess = true; // Sin restricciones
  }

  return hasAccess ? <>{children}</> : <>{fallback}</>;
}

// ─── Uso ────────────────────────────────────────
//
// <PermissionGate permission="work_orders:create">
//   <Button>Nueva Orden de Trabajo</Button>
// </PermissionGate>
//
// <PermissionGate anyOf={['reports:sales', 'reports:financial']}>
//   <SalesReportWidget />
// </PermissionGate>
//
// <PermissionGate
//   permission="admin:settings"
//   fallback={<p>No tienes acceso a configuración</p>}
// >
//   <SettingsPanel />
// </PermissionGate>
```

8.8 Testing RBAC

```
// apps/backend/test/auth/permissions.e2e-spec.ts
describe('RBAC - Control de Acceso (E2E)', () => {
  let app: INestApplication;
  let adminToken: string;
  let technicianToken: string;
  let receptionistToken: string;

  beforeAll(async () => {
    // Setup con roles predefinidos
    adminToken = await getTokenForRole(app, 'ADMIN');
    technicianToken = await getTokenForRole(app, 'TECHNICIAN');
    receptionistToken = await getTokenForRole(app, 'RECEPTIONIST');
  });

  describe('Órdenes de Trabajo', () => {
    it('Admin puede crear órdenes', async () => {
      await request(app.getHttpServer())
        .post('/api/v1/work-orders')
        .set('Authorization', `Bearer ${adminToken}`)
        .send(validWorkOrderData)
        .expect(201);
    });

    it('Recepcionista puede crear órdenes', async () => {
      await request(app.getHttpServer())
        .post('/api/v1/work-orders')
        .set('Authorization', `Bearer ${receptionistToken}`)
        .send(validWorkOrderData)
        .expect(201);
    });

    it('Técnico NO puede crear órdenes', async () => {
      await request(app.getHttpServer())
        .post('/api/v1/work-orders')
        .set('Authorization', `Bearer ${technicianToken}`)
        .send(validWorkOrderData)
        .expect(403);
    });

    it('Técnico puede ver órdenes asignadas', async () => {
      await request(app.getHttpServer())
        .get('/api/v1/work-orders')
        .set('Authorization', `Bearer ${technicianToken}`)
        .expect(200);
    });
  });

  describe('Administración', () => {
    it('Admin puede gestionar usuarios', async () => {
      await request(app.getHttpServer())
        .get('/api/v1/admin/users')
        .set('Authorization', `Bearer ${adminToken}`)
        .expect(200);
    });

    it('Recepcionista NO puede gestionar usuarios', async () => {
      await request(app.getHttpServer())
        .get('/api/v1/admin/users')
        .set('Authorization', `Bearer ${receptionistToken}`)
        .expect(403);
    });
```

```
    it('Técnico NO puede acceder a administración', async () => {
      await request(app.getHttpServer())
        .get('/api/v1/admin/settings')
        .set('Authorization', `Bearer ${technicianToken}`)
        .expect(403);
    });
  });

  describe('Invalidación de Caché', () => {
    it('Los permisos se actualizan al cambiar roles', async () => {
      // 1. Verificar que técnico no puede crear órdenes
      await request(app.getHttpServer())
        .post('/api/v1/work-orders')
        .set('Authorization', `Bearer ${technicianToken}`)
        .send(validWorkOrderData)
        .expect(403);

      // 2. Agregar permiso de crear órdenes al rol técnico
      await addPermissionToRole(app, adminToken, 'TECHNICIAN', 'work_orders:create');

      // 3. Esperar invalidación de caché
      await new Promise((r) => setTimeout(r, 100));

      // 4. Ahora el técnico SÍ puede crear
      await request(app.getHttpServer())
        .post('/api/v1/work-orders')
        .set('Authorization', `Bearer ${technicianToken}`)
        .send(validWorkOrderData)
        .expect(201);
    });
  });
});
```

9. Guías de Desarrollo

9.1 Setup Inicial del Proyecto

```bash
#!/bin/bash
# scripts/setup.sh — Setup completo del entorno de desarrollo

set -euo pipefail

echo "🔧 TallerHub — Setup de Desarrollo"
echo "==================================="

# 1. Verificar prerequisitos
echo "📋 Verificando prerequisitos..."
command -v node >/dev/null 2>&1 || { echo "❌ Node.js requerido (>= 20)"; exit 1; }
command -v pnpm >/dev/null 2>&1 || { echo "❌ pnpm requerido. Instalar: npm i -g
pnpm"; exit 1; }
command -v docker >/dev/null 2>&1 || { echo "❌ Docker requerido"; exit 1; }

NODE_VERSION=$(node -v | cut -d'v' -f2 | cut -d'.' -f1)
if [ "$NODE_VERSION" -lt 20 ]; then
  echo "❌ Node.js >= 20 requerido (actual: $(node -v))"
  exit 1
fi

echo "✅ Prerequisitos OK"

# 2. Instalar dependencias
echo "📦 Instalando dependencias..."
pnpm install

# 3. Configurar variables de entorno
echo "🔐 Configurando variables de entorno..."
if [ ! -f .env ]; then
  cp .env.example .env
  echo "⚠️  Archivo .env creado desde .env.example — REVISA los valores"
fi

if [ ! -f apps/backend/.env ]; then
  cp apps/backend/.env.example apps/backend/.env
fi

if [ ! -f apps/web/.env.local ]; then
  cp apps/web/.env.example apps/web/.env.local
fi

# 4. Levantar servicios de infraestructura
echo "🐳 Levantando PostgreSQL + Redis..."
docker compose up -d postgres redis

# Esperar a que PostgreSQL esté listo
echo "⏳ Esperando a PostgreSQL..."
until docker compose exec -T postgres pg_isready -U tallerhub > /dev/null 2>&1; do
  sleep 1
done
echo "✅ PostgreSQL listo"

# 5. Ejecutar migraciones
echo "🗃️  Ejecutando migraciones de Prisma..."
pnpm --filter @tallerhub/backend exec prisma migrate dev

# 6. Seed de datos iniciales
echo "🌱 Cargando datos iniciales (seed)..."
pnpm --filter @tallerhub/backend exec prisma db seed

# 7. Generar cliente Prisma
```

```
echo "⚙️  Generando cliente Prisma..."
pnpm --filter @tallerhub/backend exec prisma generate

# 8. Build de paquetes compartidos
echo "📦 Building paquetes compartidos..."
pnpm --filter @tallerhub/shared build

echo ""
echo "🎉 Setup completado!"
echo ""
echo "Comandos disponibles:"
echo "  pnpm dev          — Levantar todo el stack"
echo "  pnpm dev:backend  — Solo backend (NestJS)"
echo "  pnpm dev:web      — Solo frontend (Next.js)"
echo "  pnpm dev:mobile   — Solo mobile (Expo)"
echo ""
```

9.2 Comandos Útiles

```
# ─── Desarrollo ──────────────────────────────────
pnpm dev                        # Todo el stack en paralelo
pnpm dev:backend                # Solo backend (http://localhost:3001)
pnpm dev:web                    # Solo frontend (http://localhost:3000)
pnpm dev:mobile                 # Solo mobile (Expo DevTools)

# ─── Base de Datos ───────────────────────────────
pnpm db:migrate                 # Crear y ejecutar migración
pnpm db:migrate:create          # Solo crear migración sin ejecutar
pnpm db:seed                    # Ejecutar seed
pnpm db:reset                   # Reset completo (⚠️ borra datos)
pnpm db:studio                  # Abrir Prisma Studio (http://localhost:5555)
pnpm db:push                    # Push schema sin migración (dev only)

# ─── Testing ─────────────────────────────────────
pnpm test                       # Tests unitarios de todo el monorepo
pnpm test:e2e                   # Tests E2E del backend
pnpm test:cov                   # Tests con cobertura
pnpm test -- --watch            # Tests en modo watch
pnpm test -- --filter=customers # Tests de un módulo específico

# ─── Calidad de Código ───────────────────────────
pnpm lint                       # ESLint en todo el monorepo
pnpm lint:fix                   # ESLint con auto-fix
pnpm format                     # Prettier en todo el monorepo
pnpm typecheck                  # TypeScript check sin compilar

# ─── Build ───────────────────────────────────────
pnpm build                      # Build de producción completo
pnpm build:backend              # Build solo backend
pnpm build:web                  # Build solo frontend
pnpm build:shared               # Build paquetes compartidos

# ─── Docker ──────────────────────────────────────
docker compose up -d            # Levantar toda la infra
docker compose down             # Detener todo
docker compose logs -f backend  # Ver logs del backend
docker compose exec postgres psql -U tallerhub  # Acceso directo a PostgreSQL

# ─── Utilidades ──────────────────────────────────
pnpm clean                      # Limpiar node_modules y builds
pnpm deps:check                 # Verificar dependencias desactualizadas
pnpm deps:update                # Actualizar dependencias interactivamente
```

9.3 Flujo de Trabajo con Git

```
main (producción)
│
├── develop (integración)
│   │
│   ├── feature/THB-123-crear-modulo-cotizaciones
│   ├── feature/THB-124-filtros-avanzados-ordenes
│   ├── fix/THB-125-calculo-impuestos
│   └── refactor/THB-126-optimizar-queries
│
└── hotfix/THB-127-corregir-login (desde main)
```

Convenciones de Branches

Prefijo	Uso	Ejemplo
`feature/`	Nueva funcionalidad	`feature/THB-123-modulo-cotizaciones`
`fix/`	Corrección de bug	`fix/THB-125-calculo-impuestos`
`hotfix/`	Fix urgente en producción	`hotfix/THB-127-corregir-login`
`refactor/`	Refactorización	`refactor/THB-126-optimizar-queries`
`docs/`	Documentación	`docs/THB-130-api-docs`
`chore/`	Mantenimiento	`chore/THB-131-actualizar-deps`

Flujo Completo de un Feature

```
# 1. Crear branch desde develop
git checkout develop
git pull origin develop
git checkout -b feature/THB-123-modulo-cotizaciones

# 2. Desarrollar con commits convencionales
git add .
git commit -m "feat(quotes): crear schema de Prisma para cotizaciones"
git commit -m "feat(quotes): implementar CRUD de cotizaciones"
git commit -m "feat(quotes): agregar validaciones y DTOs"
git commit -m "test(quotes): agregar tests unitarios del servicio"
git commit -m "feat(quotes): implementar UI de lista de cotizaciones"

# 3. Antes del PR, verificar calidad
pnpm lint
pnpm typecheck
pnpm test
pnpm build

# 4. Push y crear Pull Request
git push origin feature/THB-123-modulo-cotizaciones
# Crear PR en GitHub: feature/THB-123 → develop

# 5. Después del merge, limpiar
git checkout develop
git pull origin develop
git branch -d feature/THB-123-modulo-cotizaciones
```

9.4 Guía: Crear un Nuevo Módulo

Pasos para crear un módulo completo de principio a fin (ejemplo: `quotes`):

Paso 1: Schema de Prisma

```
// packages/database/prisma/schema.prisma (agregar)

model Quote {
  id                String      @id @default(uuid()) @db.Uuid
  quoteNumber       String      @map("quote_number")
  tenantId          String      @map("tenant_id") @db.Uuid
  branchId          String      @map("branch_id") @db.Uuid
  customerId        String      @map("customer_id") @db.Uuid
  vehicleId         String?     @map("vehicle_id") @db.Uuid
  status            QuoteStatus @default(DRAFT)
  validUntil        DateTime    @map("valid_until")
  notes             String?
  subtotal          Decimal     @default(0) @db.Decimal(12, 2)
  taxAmount         Decimal     @default(0) @map("tax_amount") @db.Decimal(12, 2)
  discountAmount    Decimal     @default(0) @map("discount_amount") @db.Decimal(12, 2)
  totalAmount       Decimal     @default(0) @map("total_amount") @db.Decimal(12, 2)
  createdAt         DateTime    @default(now()) @map("created_at")
  updatedAt         DateTime    @updatedAt @map("updated_at")

  tenant            Tenant      @relation(fields: [tenantId], references: [id])
  branch            Branch      @relation(fields: [branchId], references: [id])
  customer          Customer    @relation(fields: [customerId], references: [id])
  vehicle           Vehicle?    @relation(fields: [vehicleId], references: [id])
  items             QuoteItem[]

  @@unique([tenantId, quoteNumber])
  @@index([tenantId, status])
  @@index([tenantId, customerId])
  @@map("quotes")
}

enum QuoteStatus {
  DRAFT
  SENT
  APPROVED
  REJECTED
  EXPIRED
  CONVERTED
}
```

```
# Ejecutar migración
pnpm db:migrate -- --name add-quotes-module
```

Paso 2: Tipos Compartidos

```
// packages/shared/src/types/quote.types.ts
export interface Quote { /* ... campos del modelo */ }
export interface CreateQuoteDto { /* ... */ }
export interface UpdateQuoteDto { /* ... */ }
```

```
// packages/shared/src/validators/quote.validators.ts
import { z } from 'zod';
export const createQuoteSchema = z.object({ /* ... */ });
```

Paso 3: Módulo Backend

```
# Crear estructura de carpetas
mkdir -p apps/backend/src/modules/quotes/{dto,entities}
```

Archivos a crear:

```
apps/backend/src/modules/quotes/
├── quotes.module.ts
├── quotes.controller.ts
├── quotes.service.ts
├── quotes.controller.spec.ts
├── quotes.service.spec.ts
├── dto/
│   ├── create-quote.dto.ts
│   ├── update-quote.dto.ts
│   └── query-quotes.dto.ts
└── entities/
    └── quote.entity.ts
```

```
// quotes.module.ts
import { Module } from '@nestjs/common';
import { QuotesController } from './quotes.controller';
import { QuotesService } from './quotes.service';

@Module({
  controllers: [QuotesController],
  providers: [QuotesService],
  exports: [QuotesService],
})
export class QuotesModule {}
```

```
// quotes.controller.ts
@Controller('quotes')
@UseGuards(TenantGuard, PermissionsGuard)
export class QuotesController {
  constructor(private readonly quotesService: QuotesService) {}

  @Post()
  @Permissions('quotes:create')
  create(@Body() dto: CreateQuoteDto, @CurrentTenant() tenant: TenantContext) {
    return this.quotesService.create(dto, tenant);
  }

  @Get()
  @Permissions('quotes:read')
  findAll(@Query() query: QueryQuotesDto, @CurrentTenant() tenant: TenantContext) {
    return this.quotesService.findAll(query, tenant);
  }

  // ... más endpoints
}
```

Paso 4: Registrar en AppModule

```
// apps/backend/src/app.module.ts — agregar al array de imports
import { QuotesModule } from './modules/quotes/quotes.module';

@Module({
  imports: [
    // ... módulos existentes
    QuotesModule,
  ],
})
export class AppModule {}
```

Paso 5: Frontend

```
mkdir -p apps/web/src/app/\(dashboard\)/quotes
mkdir -p apps/web/src/features/quotes/{components,hooks,lib}
```

Archivos principales:
- `app/(dashboard)/quotes/page.tsx` — Página principal (Server Component)
- `app/(dashboard)/quotes/[id]/page.tsx` — Detalle
- `app/(dashboard)/quotes/new/page.tsx` — Crear nueva
- `features/quotes/components/quote-table.tsx` — Tabla de cotizaciones
- `features/quotes/components/quote-form.tsx` — Formulario
- `features/quotes/hooks/useQuotes.ts` — Hook de datos

Paso 6: Tests

```
# Ejecutar tests del nuevo módulo
pnpm test -- --filter=quotes
pnpm test:e2e -- --grep="Quotes"
```

9.5 Guía: Agregar una Nueva Entidad a un Módulo Existente

```
# Ejemplo: Agregar QuoteTemplate al módulo de cotizaciones

# 1. Agregar modelo en schema.prisma
# 2. Crear migración: pnpm db:migrate -- --name add-quote-templates
# 3. Agregar tipos en packages/shared
# 4. Crear archivos en el módulo:
touch apps/backend/src/modules/quotes/dto/create-quote-template.dto.ts
touch apps/backend/src/modules/quotes/dto/update-quote-template.dto.ts
touch apps/backend/src/modules/quotes/quote-templates.controller.ts
touch apps/backend/src/modules/quotes/quote-templates.service.ts
# 5. Registrar controller y service en quotes.module.ts
# 6. Agregar tests
```

9.6 Guía: Agregar un Nuevo Endpoint

```
// Ejemplo: Agregar endpoint POST /quotes/:id/send para enviar cotización

// 1. Agregar el método en el controlador
@Post(':id/send')
@Permissions('quotes:send')
@HttpCode(HttpStatus.OK)
async sendQuote(
  @Param('id', ParseUUIDPipe) id: string,
  @Body() dto: SendQuoteDto,
  @CurrentTenant() tenant: TenantContext,
) {
  return this.quotesService.send(id, dto, tenant);
}

// 2. Implementar lógica en el servicio
async send(id: string, dto: SendQuoteDto, tenant: TenantContext) {
  const quote = await this.prisma.quote.findUnique({ where: { id } });

  if (!quote) throw new NotFoundException('Cotización no encontrada');
  if (quote.status !== 'DRAFT') {
    throw new BadRequestException('Solo se pueden enviar cotizaciones en borrador');
  }

  // Actualizar estado
  const updated = await this.prisma.quote.update({
    where: { id },
    data: { status: 'SENT', sentAt: new Date() },
  });

  // Enviar notificación por email (async)
  await this.notificationQueue.add('send-quote-email', {
    quoteId: id,
    recipientEmail: dto.email,
    tenantId: tenant.tenantId,
  });

  return updated;
}

// 3. Crear DTO de validación
// 4. Agregar test del endpoint
// 5. Actualizar constante de permisos si es necesario
```

9.7 Guía: Agregar una Nueva Pantalla (Frontend)

```
# Ejemplo: Pantalla de detalle de cotización

# 1. Crear archivo de página (Server Component)
touch apps/web/src/app/\(dashboard\)/quotes/\[id\]/page.tsx
```

```
// apps/web/src/app/(dashboard)/quotes/[id]/page.tsx
import { Suspense } from 'react';
import { notFound } from 'next/navigation';
import { apiServer } from '@/lib/api-server';
import { QuoteDetail } from '@/features/quotes/components/quote-detail';
import { QuoteDetailSkeleton } from '@/features/quotes/components/quote-detail-skelet-
on';

interface Props {
  params: { id: string };
}

export async function generateMetadata({ params }: Props) {
  const quote = await apiServer.get(`/quotes/${params.id}`).catch(() => null);
  return {
    title: quote ? `Cotización ${quote.data.quoteNumber}` : 'Cotización',
  };
}

export default async function QuoteDetailPage({ params }: Props) {
  const { data: quote } = await apiServer.get(`/quotes/${params.id}`).catch(() => not-
Found());

  return (
    <div className="container py-6 space-y-6">
      <Suspense fallback={<QuoteDetailSkeleton />}>
        <QuoteDetail quote={quote} />
      </Suspense>
    </div>
  );
}
```

9.8 Guía de Debugging

Backend

```
# Modo debug con inspector de Node.js
NODE_OPTIONS='--inspect' pnpm dev:backend
# Conectar Chrome DevTools: chrome://inspect

# Logging detallado de Prisma
DATABASE_DEBUG=true pnpm dev:backend

# Logging de queries SQL
LOG_LEVEL=debug pnpm dev:backend

# Verificar estado de Redis
docker compose exec redis redis-cli
> KEYS tenant:*          # Ver keys de tenant
> TTL <key>              # Ver TTL de un key
> GET <key>              # Ver valor
> FLUSHDB                # Limpiar caché (⚠️ dev only)

# Verificar estado de colas BullMQ
# Acceder a Bull Board: http://localhost:3001/admin/queues
```

Frontend

```
# React DevTools (instalar extensión de Chrome)
# Next.js debugging: agregar en .vscode/launch.json
{
  "type": "node",
  "request": "launch",
  "name": "Next.js Debug",
  "runtimeExecutable": "pnpm",
  "runtimeArgs": ["dev:web"],
  "port": 9230,
  "console": "integratedTerminal"
}
```

Mobile

```
# React Native Debugger
npx react-devtools

# Logs de Expo
npx expo start --clear  # Limpiar caché de Metro
# Shake device → Debug Remote JS

# Verificar conectividad API desde dispositivo
# Usar ngrok para exponer backend local:
ngrok http 3001
```

9.9 Mejores Prácticas

Performance

```
// ✅ Seleccionar solo campos necesarios
const customers = await this.prisma.customer.findMany({
  where: { tenantId },
  select: { id: true, firstName: true, lastName: true, email: true },
});

// ❌ Traer todas las relaciones sin necesidad
const customers = await this.prisma.customer.findMany({
  where: { tenantId },
  include: { vehicles: true, workOrders: true, quotes: true },
});
```

```
// ✅ Paginación con cursor (eficiente en grandes datasets)
const results = await this.prisma.workOrder.findMany({
  where: { tenantId },
  take: limit + 1,
  cursor: cursor ? { id: cursor } : undefined,
  orderBy: { createdAt: 'desc' },
});

// ❌ Paginación con offset (lenta en páginas altas)
const results = await this.prisma.workOrder.findMany({
  where: { tenantId },
  skip: (page - 1) * limit,
  take: limit,
});
```

```
// ✅ Operaciones batch cuando sea posible
await this.prisma.workOrderItem.createMany({
  data: items.map((item) => ({ ...item, workOrderId })),
});

// ❌ Inserciones una por una en loop
for (const item of items) {
  await this.prisma.workOrderItem.create({
    data: { ...item, workOrderId },
  });
}
```

Seguridad

```
// ✅ Siempre validar datos de entrada con DTOs
@Post()
create(@Body(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true }))
dto: CreateDto)

// ✅ Sanitizar outputs — nunca exponer datos internos
return plainToInstance(CustomerResponseDto, customer, {
  excludeExtraneousValues: true,
});

// ✅ Rate limiting en endpoints sensibles
@Throttle({ default: { limit: 5, ttl: 60000 } })
@Post('auth/login')

// ✅ Auditar operaciones críticas
await this.auditService.log({
  action: 'WORK_ORDER_STATUS_CHANGE',
  entityType: 'WorkOrder',
  entityId: workOrder.id,
  oldValue: { status: oldStatus },
  newValue: { status: newStatus },
  userId: tenant.userId,
  tenantId: tenant.tenantId,
});
```

Mantenibilidad

```
// ✅ Un servicio por responsabilidad
@Injectable()
export class WorkOrderCalculationService {
  calculateTotals(items: WorkOrderItem[]): OrderTotals { /* ... */ }
  calculateTax(subtotal: number, taxRate: number): number { /* ... */ }
  applyDiscount(total: number, discount: Discount): number { /* ... */ }
}

// ✅ Constantes en lugar de magic numbers
const MAX_ITEMS_PER_ORDER = 100;
const DEFAULT_TAX_RATE = 0.19; // IVA Chile
const CACHE_TTL_SECONDS = 300;

// ✅ Manejo explícito de errores de negocio
export class InsufficientStockError extends BadRequestException {
  constructor(productId: string, requested: number, available: number) {
    super(
      `Stock insuficiente para producto ${productId}: ` +
      `solicitado=${requested}, disponible=${available}`,
    );
  }
}
```

Apéndice A: Checklist de Nuevo Módulo

#	Tarea	Archivo/Ubicación
1	Modelo en Prisma schema	`packages/database/prisma/schema.prisma`
2	Migración	`pnpm db:migrate -- --name <nombre>`
3	Tipos compartidos	`packages/shared/src/types/<modulo>.types.ts`
4	Validadores Zod	`packages/shared/src/validators/<modulo>.validators.ts`
5	DTOs del backend	`apps/backend/src/modules/<modulo>/dto/`
6	Service + Controller	`apps/backend/src/modules/<modulo>/`
7	Registrar en AppModule	`apps/backend/src/app.module.ts`
8	Tests unitarios	`apps/backend/src/modules/<modulo>/*.spec.ts`
9	Tests E2E	`apps/backend/test/<modulo>/`
10	Permisos	`packages/shared/src/constants/permissions.ts`
11	Seed de roles/permisos	`packages/database/prisma/seed.ts`
12	Páginas frontend	`apps/web/src/app/(dashboard)/<modulo>/`
13	Feature components	`apps/web/src/features/<modulo>/`
14	Hook de datos	`apps/web/src/features/<modulo>/hooks/`
15	Pantallas mobile	`apps/mobile/src/features/<modulo>/`
16	Navegación mobile	`apps/mobile/src/features/<modulo>/navigation/`

#	Tarea	Archivo/Ubicación
17	Documentación API	Swagger decorators en controller

Apéndice B: Variables de Entorno Completas

Variable	Descripción	Ejemplo
`DATABASE_URL`	Connection string PostgreSQL	`postgresql://user:pass@localhost:5432/tallerhub`
`REDIS_URL`	Connection string Redis	`redis://localhost:6379`
`JWT_SECRET`	Secreto para firmar JWT	`super-secret-key-min-32-chars!!!`
`JWT_REFRESH_SECRET`	Secreto para refresh tokens	`another-secret-key-min-32-chars!`
`JWT_EXPIRATION`	Expiración del access token	`15m`
`JWT_REFRESH_EXPIRATION`	Expiración del refresh token	`7d`
`CORS_ORIGINS`	Orígenes permitidos (CSV)	`http://localhost:3000,https://app.tallerhub.com`
`SMTP_HOST`	Servidor SMTP	`smtp.resend.com`
`SMTP_PORT`	Puerto SMTP	`587`
`SMTP_USER`	Usuario SMTP	`resend`
`SMTP_PASS`	Contraseña SMTP	`re_xxxxx`
`S3_BUCKET`	Bucket para archivos	`tallerhub-uploads`
`S3_REGION`	Región de S3	`us-east-1`
`S3_ACCESS_KEY`	Access key de AWS	`AKIA...`
`S3_SECRET_KEY`	Secret key de AWS	`xxxxx`
`SENTRY_DSN`	DSN de Sentry	`https://xxx@sentry.io/xxx`
`LOG_LEVEL`	Nivel de logging	`debug` / `info` / `warn` / `error`
`NODE_ENV`	Entorno de ejecución	`development` / `production`
`PORT`	Puerto del backend	`3001`

> **Documento generado para TallerHub v1.0 — Fase 2: Implementación Técnica**
> Última actualización: Marzo 2026
> Para uso interno del equipo de desarrollo.